LLMs for Fun

From Zero to a State-of-the-Art LLM's Inference Engine

PyTorch, Rust, C++, CUDA, and correctness-first systems design

Conceptual guide. This book is the mathematical companion to the repository's conceptual guide. It follows the guide's phases, invariants, tensor names, cache semantics, CLI behaviour, artifact layout, validation expectations, and roadmap order without treating any single prose file as a substitute for the executable specifications.

Complete LaTeX source build
April 2026

Copyright and licence placeholder

Copyright and licence terms should be supplied by the project owner before publication. Until then, this manuscript is a technical design and education draft aligned with the repository's conceptual guide.

Preface

This book explains how to build an LLM inference engine from first principles while preserving the repository's shared contracts. It is not a training manual and it is not an implementation dump. It is a mathematical and systems design book that connects equations, tensor shapes, artifact files, cache semantics, language interfaces, validation vectors, and eventually CUDA kernels.

Roadmap invariant

Phase 0 is GPT-2, CPU-first, correctness-first, with PyTorch as the reference engine. C++ and Rust consume the same converted artifacts and must match PyTorch within explicit tolerance. Later phases add LLaMA-style support, RoPE, RMSNorm, gated MLPs, MQA, GQA, sliding-window cache policies, modern dense families, hybrid or recurrent blocks, and CUDA acceleration.

How to use this book

Read Parts I and II for foundations and architecture. Part III is the phase-0 implementation spine. Parts IV and V explain roadmap extensions. Part VI teaches CUDA acceleration after CPU correctness is established. Part VII gives training context only to distinguish inference from training.

Every major chapter includes intuition, a formal or contract statement, implementation interpretation, verification strategy, common failure modes, and exercises with hints.

Notation

All tensor equations use mathematical indexing. Physical storage may use a different stride order if the implementation preserves the same logical indices.

- B is batch size, T is the current input length, S is the visible cache length, L is the number of transformer blocks, V is vocabulary size, and P is the maximum learned-position table length.
- H is hidden size, F is feed-forward width, H_q is the number of query heads, H_{kv} is the number of key/value heads, and $d_h = H/H_q$ when attention head dimensions are uniform.
- $I \in \mathbb{N}^{B\times T}$ denotes token ids. Hidden states are $X \in \mathbb{R}^{B\times T\times H}$, logits are $Z \in \mathbb{R}^{B\times T\times V}$, and block outputs are written $X^{(\ell)}$.
- Query tensors have logical shape $Q \in \mathbb{R}^{B\times H_q\times T\times d_h}$. Cached keys and values for layer ℓ have logical shape $K_\ell, V_\ell \in \mathbb{R}^{B\times H_{kv}\times S\times d_h}$.
- For GQA, $g = H_q/H_{kv}$ must be an integer and query head h reads key/value head $\lfloor h/g \rfloor$. Phase-0 GPT-2 is the MHA case $H_{kv} = H_q$.
- The canonical KV-cache logical layout is $[B, H_{kv}, S, d_h]$, ordered as batch, key/value head, sequence, and head dimension. This is a logical contract, not a mandatory physical stride order.

Project assumptions

This repository is milestone 0.1 material: specifications and documentation exist, while implementation directories and model artifacts are target structure. This book therefore gives pseudocode, interfaces, validation procedures, and mathematical derivations rather than pretending that repository code already exists.

Key invariants used throughout

- Converted artifacts use internal tensor names, not upstream checkpoint names.
- `weights.index.json` offsets and byte counts are authoritative.
- Phase-0 artifact support is `float32`.
- GPT-2 and MHA require `num_key_value_heads == num_attention_heads`.
- KV-cache logical layout is $[B, H_{kv}, S, d_h]$.
- Model-math validation is separate from tokenizer validation.
- Cache equivalence is mandatory.
- Benchmarking is not validation.

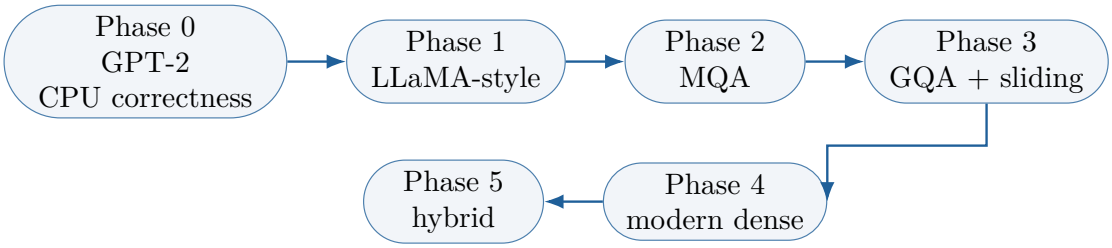

Figure 1: Repository roadmap: correctness-first GPT-2, then modern transformer and hybrid phases.

Build assumptions

The book is a standard LaTeX project built with `pdflatex` and BibTeX-compatible bibliography tooling. The engine described assumes a Linux-oriented development environment, modern Python, modern PyTorch, Rust 2024 edition, C++20 or later, CMake or equivalent build tooling, and CUDA Toolkit documentation checked at implementation time [PyTorch Contributors, 2026, The Rust Project Developers, 2025, International Organization for Standardization, 2020, Kitware, 2026, NVIDIA Corporation, 2026d,b,a].

Contents

I Foundations 1

1 Why build an LLM inference engine? 2

2 Mathematical notation for tensors and shapes 4

3 Linear algebra for inference engineers 6

4 Probability, logits, softmax, and sampling 8

5 Numerical precision and error tolerances 10

6 PyTorch for reference implementations 12

7 Rust for safe systems inference 14

8 C++ for portable high-performance inference 15

9 CUDA programming model, introduced early 17

10 Testing, verification, golden vectors, and reproducibility 19

II Engine architecture 21

11 Shared contracts and why the repo is contract-first 22

12 Model configuration 24

13 Tensor naming and weight artifact design 26

14 SafeTensors and conversion into internal artifacts 28

15 Tokenizers and embedders 30

16 Decoder-only model architecture 31

17 Prefill, decode, and generation loops 32

18 KV-cache abstraction and cache policies 34

19 CLI, validation, and benchmarking contracts 36

20 High-level PyTorch/Rust/C++ interface design 38

III Phase 0: GPT-2 CPU-first correctness 40

21 GPT-2 architecture 41

22 Token and learned position embeddings 43

23 LayerNorm 45

24 Causal self-attention 47

25 GELU MLP 49

26 Transformer block composition 51

27 Final normalisation and language-modelling head 52

28 PyTorch reference engine 54

29 Exporting golden vectors 56

30 C++ correctness engine 58

31 Rust correctness engine 60

32 Cross-language validation 62

33 Cache-equivalence testing 63

34 Baseline benchmarking 64

IV Modern transformer phases 65

35 LLaMA-style architectural changes 66

36 RoPE 67

37 RMSNorm 69

38 Gated MLPs: SwiGLU and GeGLU 71

39 MQA 73

40 GQA 74

41 Sliding-window attention and cache policies 76

42 Modern dense model-family adapters 78

43 Qwen/Gemma/Mistral-style implementation considerations 79

44 Long-context validation 80

45 Quantisation overview, marked as later work 81

V Hybrid and recurrent phases 82

46 Why hybrid token mixers matter 83

47 State-space and recurrent intuition 84

48 Mamba-style blocks 86

49 Recurrent state versus KV-cache 87

50 Interface design for heterogeneous layers 88

51 Validation for hybrid models 89

VI CUDA acceleration and performance 90

52 From CPU kernels to CUDA kernels 91

53 Memory layout and tensor storage 93

54 CUDA matmul concepts 95

55 LayerNorm and RMSNorm kernels 97

56 Softmax kernels 98

57 Attention kernels 100

58 Prefill kernels 101

59 Decode kernels 102

60 KV-cache layout on GPU 103

61 Kernel fusion 104

62 Tensor Cores and mixed precision 105

63 CUDA Graphs and launch overhead 106

64 Profiling workflow 107

65 Benchmark interpretation 108

66 Optional architecture notes for RTX 40-series and RTX 5060-style GPUs 109

VII Training for context 111

67 What training would add 112

68 Language-modelling loss 113

69 Backpropagation overview 115

70 Optimisers and state 116

71 Why inference is a different engineering problem 117

VIII Appendices 119

A LaTeX notation reference 120

B Tensor shape cheat sheet 122

C CUDA technical details 124

D Rust ownership patterns for inference engines 125

E C++ memory and RAII patterns 126

F PyTorch reference implementation patterns **127**

G Artifact validation checklist **129**

H Golden-vector generation checklist **131**

I Benchmark checklist **133**

J Glossary **135**

K Bibliography notes **137**

Part I

Foundations

Chapter 1

Why build an LLM inference engine?

An inference engine is a mathematical model wrapped in a file-format contract, a runtime architecture, and a validation discipline. This book starts with GPT-2 because it is compact enough to audit yet exercises the core decoder-only path. [Vaswani et al., 2017, Radford et al., 2018, 2019]

Intuition

The first useful engine is the one whose behaviour is explainable and reproducible across runtimes.

Mathematical or contract statement

For token ids $x_{1:T}$, inference computes logits $z_T = f_\theta(x_{1:T}) \in \mathbb{R}^V$. The project target is tolerance-based agreement across PyTorch, C++, and Rust over the same converted artifacts.

Implementation interpretation

Build the narrow complete slice first: converter, PyTorch reference, golden vectors, C++ engine, Rust engine, shared validation, then benchmark baseline. CUDA is studied early but implemented later.

How to verify this works

- Check shapes and dtypes at the boundary.
- Compare deterministic outputs against PyTorch golden vectors.
- Record tolerance, seed, model artifact, and backend.
- Do not benchmark until validation passes.

Common failure modes

- Letting framework defaults choose dtype or layout.
- Testing only a batch size of one.
- Depending on upstream checkpoint names in runtime code.
- Treating benchmark output as validation.

Exercises

E1.1. State the central invariant in one sentence.
Hint. Use the conceptual guide vocabulary.

E1.2. Construct a toy example with small shapes.
Hint. Use $B = 2$, $T = 3$, $H = 8$ where possible.

E1.3. Name a validation test that would catch a plausible bug.
Hint. Prefer generated golden vectors to handwritten long arrays.

Chapter 2

Mathematical notation for tensors and shapes

Most inference bugs begin as shape bugs. This chapter fixes the notation for tensors, broadcasting, layout, and cache dimensions.

Symbol	Toy value	Meaning
B	1 or 2	Batch size.
T	3 or 4	Current input sequence length.
S	accumulated	Visible cache sequence length.
L	2 or 12	Number of transformer blocks.
P	16 or 1024	Maximum learned-position table length.
H	8 or 16	Hidden size.
F	32 or 64	Feed-forward intermediate width.
H_q	2 or 4	Query attention heads.
H_{kv}	H_q in phase 0	Key/value heads.
d_h	H/H_q	Per-head dimension.
g	H_q/H_{kv}	Query heads per key/value head for GQA.
V	toy vocab or 50257	Vocabulary size.

Table 2.1: Shape notation used throughout the book.

Intuition

A shape is a precondition, not a comment.

Mathematical or contract statement

Token ids are $I \in \mathbb{N}^{B\times T}$. Hidden states are $X \in \mathbb{R}^{B\times T\times H}$. Query tensors are $Q \in \mathbb{R}^{B\times H_q\times T\times d_h}$. For layer ℓ, cached keys and values are logically $K_\ell, V_\ell \in \mathbb{R}^{B\times H_{kv}\times S\times d_h}$. A causal mask $M \in \mathbb{R}^{T\times S}$ uses finite values for visible keys and $-\infty$ for hidden keys.

Implementation interpretation

Store shape, dtype, device, and layout metadata on tensor objects. Validate $H \bmod H_q = 0$, $H_q \bmod H_{kv} = 0$, and cache sequence bounds before execution. Keep logical layout distinct from physical storage.

How to verify this works

- Check shapes and dtypes at the boundary.
- Compare deterministic outputs against PyTorch golden vectors.
- Record tolerance, seed, model artifact, and backend.
- Do not benchmark until validation passes.

Common failure modes

- Letting framework defaults choose dtype or layout.
- Testing only a batch size of one.
- Depending on upstream checkpoint names in runtime code.
- Treating benchmark output as validation.

Exercises

E2.1. State the central invariant in one sentence.
Hint. Use the conceptual guide vocabulary.

E2.2. Construct a toy example with small shapes.
Hint. Use $B = 2$, $T = 3$, $H = 8$ where possible.

E2.3. Name a validation test that would catch a plausible bug.
Hint. Prefer generated golden vectors to handwritten long arrays.

Chapter 3

Linear algebra for inference engineers

Affine maps and matrix multiplication power attention projections, MLPs, and the final language-modelling head.

Intuition

Algebra tells us what to compute; layout and bandwidth tell us what it costs.

Mathematical or contract statement

For $X \in \mathbb{R}^{N \times H}$, $W \in \mathbb{R}^{O \times H}$, and $b \in \mathbb{R}^{O}$, a linear layer computes $Y = XW^{\top} + \mathbf{1}_N b^{\top} \in \mathbb{R}^{N \times O}$. For a hidden tensor $X \in \mathbb{R}^{B \times T \times H}$, use $N = BT$ or apply the same affine map independently to each (b, t) row.

Implementation interpretation

Implement a clear CPU reference first. Use non-square toy matrices so transposition errors cannot hide.

How to verify this works

- Check shapes and dtypes at the boundary.
- Compare deterministic outputs against PyTorch golden vectors.
- Record tolerance, seed, model artifact, and backend.
- Do not benchmark until validation passes.

Common failure modes

- Letting framework defaults choose dtype or layout.
- Testing only a batch size of one.
- Depending on upstream checkpoint names in runtime code.
- Treating benchmark output as validation.

Exercises

E3.1. State the central invariant in one sentence.
Hint. Use the conceptual guide vocabulary.

E3.2. Construct a toy example with small shapes.
Hint. Use $B = 2$, $T = 3$, $H = 8$ where possible.

E3.3. Name a validation test that would catch a plausible bug.
Hint. Prefer generated golden vectors to handwritten long arrays.

Chapter 4

Probability, logits, softmax, and sampling

The model produces logits. Samplers turn logits into token ids using greedy, temperature, top-k, or top-p policies.

Intuition

Logits are scores; softmax is normalisation; sampling is policy.

Mathematical or contract statement

Stable softmax uses $m = \max_i z_i$ and $p_i = \exp(z_i - m)/\sum_j \exp(z_j - m)$. Greedy sampling returns $\arg\max_i z_i$. Temperature sampling applies $\mathrm{softmax}(z/\tau)$ for $\tau > 0$. Top-k masks all but the k largest logits; top-p keeps the smallest descending-probability prefix whose cumulative probability is at least p.

Implementation interpretation

Validate logits and greedy token ids for model math. Stochastic sampling needs explicit seed, filter order, and categorical draw.

How to verify this works

- Check shapes and dtypes at the boundary.
- Compare deterministic outputs against PyTorch golden vectors.
- Record tolerance, seed, model artifact, and backend.
- Do not benchmark until validation passes.

Common failure modes

- Letting framework defaults choose dtype or layout.
- Testing only a batch size of one.
- Depending on upstream checkpoint names in runtime code.
- Treating benchmark output as validation.

Exercises

E4.1. State the central invariant in one sentence.
Hint. Use the conceptual guide vocabulary.

E4.2. Construct a toy example with small shapes.
Hint. Use $B = 2$, $T = 3$, $H = 8$ where possible.

E4.3. Name a validation test that would catch a plausible bug.
Hint. Prefer generated golden vectors to handwritten long arrays.

Chapter 5

Numerical precision and error tolerances

Phase 0 artifacts are `float32`. Later half precision, bfloat16, TensorFloat-32, or quantisation require explicit new contracts.

Intuition

Floating-point equality is a test policy, not a wish.

Mathematical or contract statement

A tensor comparison passes when $|a_i - b_i| \leq \text{atol} + \text{rtol}|b_i|$ for all entries after shape and dtype checks pass. Validators should report $\max_i |a_i - b_i|$, the largest relative error over nonzero references, and the first failing index. Phase-0 hidden states and logits use explicit tolerances, while greedy ids match exactly.

Implementation interpretation

Carry dtype through loaders, tensors, ops, exporters, and validators. Report max absolute error, max relative error, and first failing index.

How to verify this works

- Check shapes and dtypes at the boundary.
- Compare deterministic outputs against PyTorch golden vectors.
- Record tolerance, seed, model artifact, and backend.
- Do not benchmark until validation passes.

Common failure modes

- Letting framework defaults choose dtype or layout.
- Testing only a batch size of one.
- Depending on upstream checkpoint names in runtime code.
- Treating benchmark output as validation.

Exercises

E5.1. State the central invariant in one sentence.
Hint. Use the conceptual guide vocabulary.

E5.2. Construct a toy example with small shapes.
Hint. Use $B = 2$, $T = 3$, $H = 8$ where possible.

E5.3. Name a validation test that would catch a plausible bug.
Hint. Prefer generated golden vectors to handwritten long arrays.

Chapter 6

PyTorch for reference implementations

PyTorch is the phase-0 correctness oracle and golden-vector exporter, but it must consume the same internal artifact format as C++ and Rust. [PyTorch Contributors, 2026]

Intuition

The best reference implementation is boring and inspectable.

Mathematical or contract statement

The reference exports selected tensors: input ids, position ids, embedding output, block outputs, final hidden, logits, prefill logits, decode logits, and generated ids.

Implementation interpretation

Construct PyTorch modules from internal tensor names. Do not validate against an upstream model object that bypasses artifact conversion.

How to verify this works

- Check shapes and dtypes at the boundary.
- Compare deterministic outputs against PyTorch golden vectors.
- Record tolerance, seed, model artifact, and backend.
- Do not benchmark until validation passes.

Common failure modes

- Letting framework defaults choose dtype or layout.
- Testing only a batch size of one.
- Depending on upstream checkpoint names in runtime code.
- Treating benchmark output as validation.

Exercises

E6.1. State the central invariant in one sentence.
Hint. Use the conceptual guide vocabulary.

E6.2. Construct a toy example with small shapes.
Hint. Use $B = 2$, $T = 3$, $H = 8$ where possible.

E6.3. Name a validation test that would catch a plausible bug.
Hint. Prefer generated golden vectors to handwritten long arrays.

Chapter 7

Rust for safe systems inference

Rust expresses ownership of buffers, tensor views, memory maps, and cache state. [Klabnik et al., 2026, The Rust Project Developers, 2025]

Intuition

Make invalid memory states hard to represent.

Mathematical or contract statement

A Rust tensor view should carry dtype, shape, and a lifetime tied to backing storage. Fallible artifact loading returns typed errors.

Implementation interpretation

Organise crates around core contracts, ops, model adapters, CLI, and tests. Avoid unsafe kernels until the safe correctness path is proven.

How to verify this works

- Check shapes and dtypes at the boundary.
- Compare deterministic outputs against PyTorch golden vectors.
- Record tolerance, seed, model artifact, and backend.
- Do not benchmark until validation passes.

Common failure modes

- Letting framework defaults choose dtype or layout.
- Testing only a batch size of one.
- Depending on upstream checkpoint names in runtime code.
- Treating benchmark output as validation.

Exercises

E7.1. State the central invariant in one sentence.
Hint. Use the conceptual guide vocabulary.

E7.2. Construct a toy example with small shapes.
Hint. Use $B = 2$, $T = 3$, $H = 8$ where possible.

E7.3. Name a validation test that would catch a plausible bug.
Hint. Prefer generated golden vectors to handwritten long arrays.

Chapter 8

C++ for portable high-performance inference

C++20 or later is a natural fit for CPU correctness and later high-performance paths. [International Organization for Standardization, 2020, Kitware, 2026]

Intuition

C++ should be explicit before it is clever.

Mathematical or contract statement

A C++ tensor view is a pointer or span plus shape, dtype, layout, and owner context. RAII owns buffers and mappings.

Implementation interpretation

Keep scalar CPU kernels simple. Add SIMD, threading, and CUDA interop only after PyTorch parity.

How to verify this works

- Check shapes and dtypes at the boundary.
- Compare deterministic outputs against PyTorch golden vectors.
- Record tolerance, seed, model artifact, and backend.
- Do not benchmark until validation passes.

Common failure modes

- Letting framework defaults choose dtype or layout.
- Testing only a batch size of one.
- Depending on upstream checkpoint names in runtime code.
- Treating benchmark output as validation.

Exercises

E8.1. State the central invariant in one sentence.
Hint. Use the conceptual guide vocabulary.

E8.2. Construct a toy example with small shapes.
Hint. Use $B = 2$, $T = 3$, $H = 8$ where possible.

E8.3. Name a validation test that would catch a plausible bug.
Hint. Prefer generated golden vectors to handwritten long arrays.

Chapter 9

CUDA programming model, introduced early

CUDA is not a phase-0 deliverable, but its execution and memory model should shape interface design from the start. [NVIDIA Corporation, 2026d,b,a]

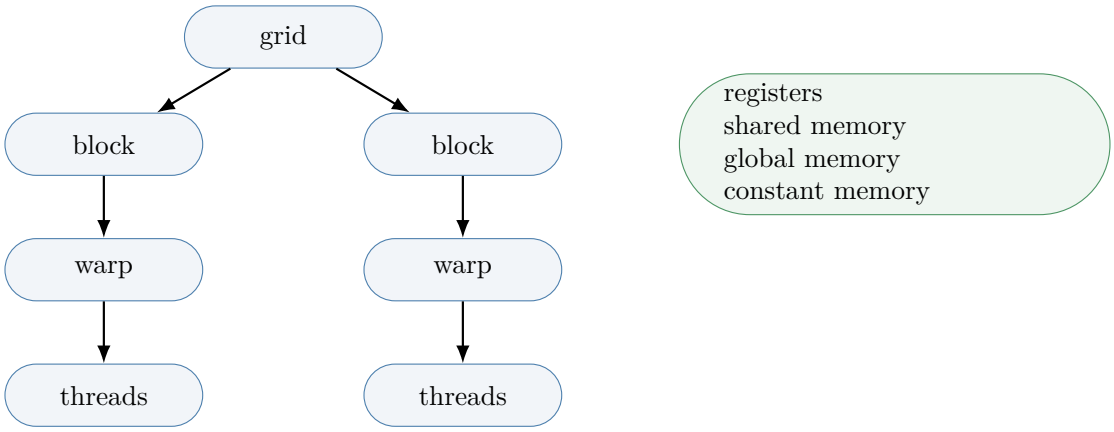

Figure 9.1: CUDA execution and memory hierarchy.

Intuition

A GPU is a throughput machine whose performance depends on parallelism and data movement.

Mathematical or contract statement

CUDA launches grids of blocks of threads. Warps execute together, and memory hierarchy includes registers, shared memory, global memory, and constant memory.

Implementation interpretation

Keep `Device`, layout, and profiling concepts in the design even while phase 0 is CPU-only.

How to verify this works

- Check shapes and dtypes at the boundary.
- Compare deterministic outputs against PyTorch golden vectors.
- Record tolerance, seed, model artifact, and backend.

- Do not benchmark until validation passes.

Common failure modes

- Letting framework defaults choose dtype or layout.
- Testing only a batch size of one.
- Depending on upstream checkpoint names in runtime code.
- Treating benchmark output as validation.

Exercises

E9.1. State the central invariant in one sentence.
Hint. Use the conceptual guide vocabulary.

E9.2. Construct a toy example with small shapes.
Hint. Use $B = 2$, $T = 3$, $H = 8$ where possible.

E9.3. Name a validation test that would catch a plausible bug.
Hint. Prefer generated golden vectors to handwritten long arrays.

Chapter 10

Testing, verification, golden vectors, and reproducibility

Validation is part of the engine contract. PyTorch exports vectors; C++ and Rust consume them.

Intuition

A good vector localises a bug.

Mathematical or contract statement

JSON smoke vectors and NPZ tensor dumps carry model family, model name, input ids, dtype, tolerance, and named arrays. Model-math validation is separate from tokenizer validation.

Implementation interpretation

Generate vectors from PyTorch; do not paste long arrays into LaTeX or source comments. Benchmark suites are separate from correctness vectors.

How to verify this works

- Check shapes and dtypes at the boundary.
- Compare deterministic outputs against PyTorch golden vectors.
- Record tolerance, seed, model artifact, and backend.
- Do not benchmark until validation passes.

Common failure modes

- Letting framework defaults choose dtype or layout.
- Testing only a batch size of one.
- Depending on upstream checkpoint names in runtime code.
- Treating benchmark output as validation.

Exercises

E10.1. State the central invariant in one sentence.
Hint. Use the conceptual guide vocabulary.

E10.2. Construct a toy example with small shapes.

Hint. Use $B = 2$, $T = 3$, $H = 8$ where possible.

E10.3. Name a validation test that would catch a plausible bug.

Hint. Prefer generated golden vectors to handwritten long arrays.

Part II

Engine architecture

Chapter 11

Shared contracts and why the repo is contract-first

The repository is organised around shared contracts rather than independent language ports. The specs define behaviour; artifacts carry data; implementations consume the same contract.

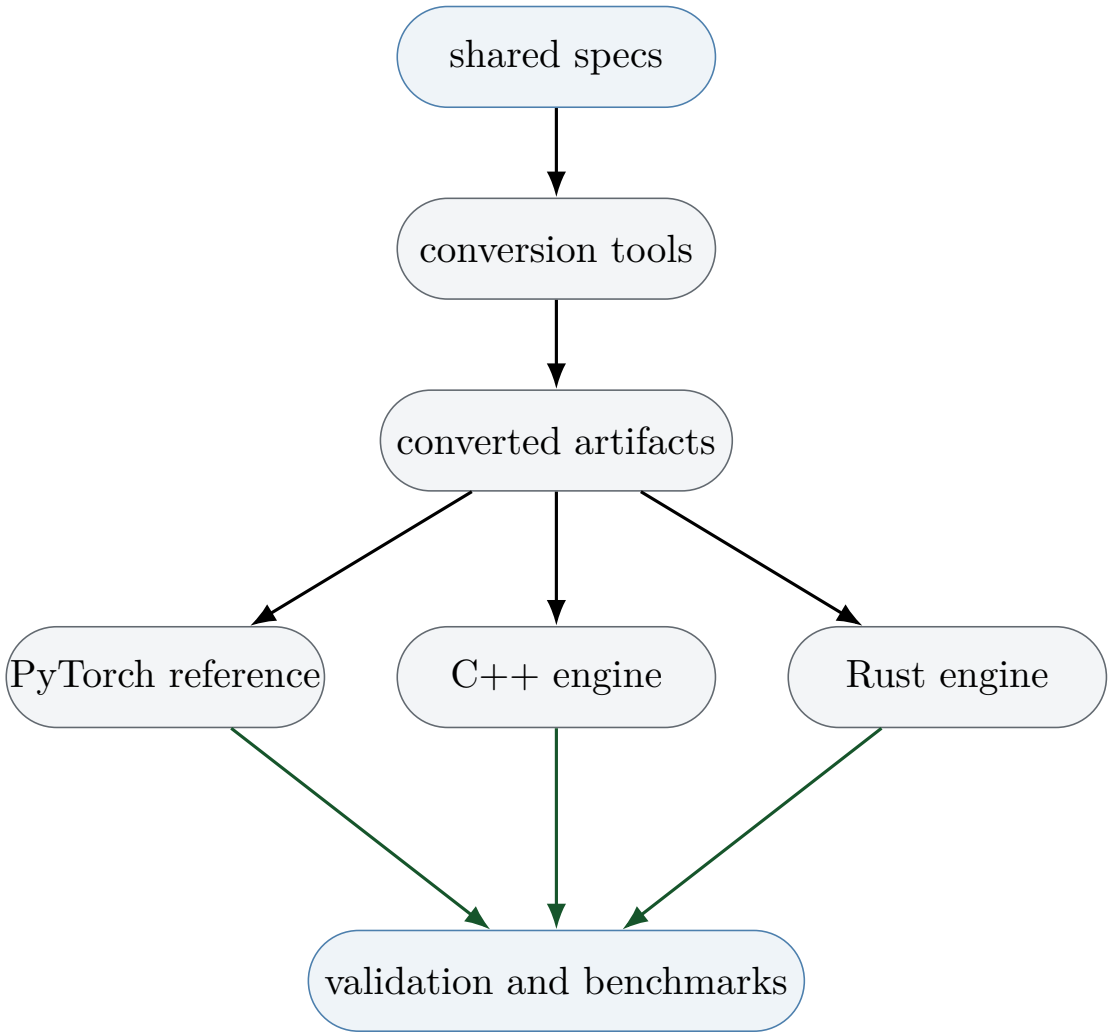

Figure 11.1: Contract-first architecture: three runtimes consume one artifact and one validation contract.

Intuition

A shared contract is what makes cross-language comparison meaningful.

Mathematical or contract statement

The shared contract includes model config, tensor names, artifact layout, cache semantics, test-vector schema, CLI output, and benchmark separation.

Implementation interpretation

Put artifact validation before model execution. A runtime that cannot prove the artifact is valid should not produce logits.

How to verify this works

- Check shapes and dtypes at the boundary.
- Compare deterministic outputs against PyTorch golden vectors.
- Record tolerance, seed, model artifact, and backend.
- Do not benchmark until validation passes.

Common failure modes

- Letting framework defaults choose dtype or layout.
- Testing only a batch size of one.
- Depending on upstream checkpoint names in runtime code.
- Treating benchmark output as validation.

Exercises

E11.1. State the central invariant in one sentence.
Hint. Use the conceptual guide vocabulary.

E11.2. Construct a toy example with small shapes.
Hint. Use $B = 2$, $T = 3$, $H = 8$ where possible.

E11.3. Name a validation test that would catch a plausible bug.
Hint. Prefer generated golden vectors to handwritten long arrays.

Chapter 12

Model configuration

The model config is internal to the engine. It is not a Hugging Face config.

Field group	Required semantics
Identity	`model_family`, `model_name`, `architecture`.
Sizes	`vocab_size`, `max_position_embeddings`, `hidden_size`, layers, heads, KV heads, intermediate size.
Module choices	Activation, norm type, positional encoding, MLP type, attention type.
Flags	Attention bias, MLP bias, tied embeddings.
Tokens and dtype	BOS, EOS, and phase-0 `float32`.
Nullable later fields	`rope_theta`, `rope_scaling`, initializer range.

Table 12.1: Model configuration groups.

Intuition

Configuration is the type signature of the model.

Mathematical or contract statement

Phase-0 GPT-2 requires `architecture=decoder_only`, `float32`, learned absolute positions, MHA, and `num_key_value_heads == num_attention_heads`.

Implementation interpretation

Parse into a strongly checked `ModelConfig`. Compute derived values such as `head_dim` only after validation.

How to verify this works

- Check shapes and dtypes at the boundary.
- Compare deterministic outputs against PyTorch golden vectors.
- Record tolerance, seed, model artifact, and backend.
- Do not benchmark until validation passes.

Common failure modes

- Letting framework defaults choose dtype or layout.
- Testing only a batch size of one.

- Depending on upstream checkpoint names in runtime code.
- Treating benchmark output as validation.

Exercises

E12.1. State the central invariant in one sentence.
Hint. Use the conceptual guide vocabulary.

E12.2. Construct a toy example with small shapes.
Hint. Use $B = 2$, $T = 3$, $H = 8$ where possible.

E12.3. Name a validation test that would catch a plausible bug.
Hint. Prefer generated golden vectors to handwritten long arrays.

Chapter 13

Tensor naming and weight artifact design

Converted artifacts use canonical internal tensor names. Runtime loaders must not depend on upstream checkpoint names.

Table 13.1: Required GPT-2 internal tensor names.

Tensor	Shape	Role
`tok_embeddings.weight`	$[V, H]$	Token table
`pos_embeddings.weight`	$[P, H]$	Positions
`blocks.i.ln_1.weight` and `bias`	$[H]$	First norm
`blocks.i.attn.qkv.weight`	$[3H, H]$	Fused QKV
`blocks.i.attn.qkv.bias`	$[3H]$	QKV bias
`blocks.i.attn.out_proj.weight`	$[H, H]$	Output projection
`blocks.i.attn.out_proj.bias`	$[H]$	Output bias
`blocks.i.ln_2.weight` and `bias`	$[H]$	Second norm
`blocks.i.mlp.fc_in.weight`	$[F, H]$	MLP expansion
`blocks.i.mlp.fc_in.bias`	$[F]$	MLP bias
`blocks.i.mlp.fc_out.weight`	$[H, F]$	MLP contraction
`blocks.i.mlp.fc_out.bias`	$[H]$	MLP bias
`ln_f.weight` and `bias`	$[H]$	Final norm
`lm_head.weight`	$[V, H]$	Explicit LM head

Intuition

Names are part of the engine ABI.

Mathematical or contract statement

GPT-2 uses top-level prefixes `tok_embeddings`, `pos_embeddings`, `blocks`, `ln_f`, and `lm_head`.

Implementation interpretation

The converter owns upstream-name knowledge. Runtime loaders use only internal names and required shapes.

How to verify this works

- Check shapes and dtypes at the boundary.
- Compare deterministic outputs against PyTorch golden vectors.

- Record tolerance, seed, model artifact, and backend.
- Do not benchmark until validation passes.

Common failure modes

- Letting framework defaults choose dtype or layout.
- Testing only a batch size of one.
- Depending on upstream checkpoint names in runtime code.
- Treating benchmark output as validation.

Exercises

E13.1. State the central invariant in one sentence.
Hint. Use the conceptual guide vocabulary.

E13.2. Construct a toy example with small shapes.
Hint. Use $B = 2$, $T = 3$, $H = 8$ where possible.

E13.3. Name a validation test that would catch a plausible bug.
Hint. Prefer generated golden vectors to handwritten long arrays.

Chapter 14

SafeTensors and conversion into internal artifacts

The project supports direct SafeTensors loading for inspection and conversion from SafeTensors or upstream checkpoints into internal artifacts. [Hugging Face, 2026]

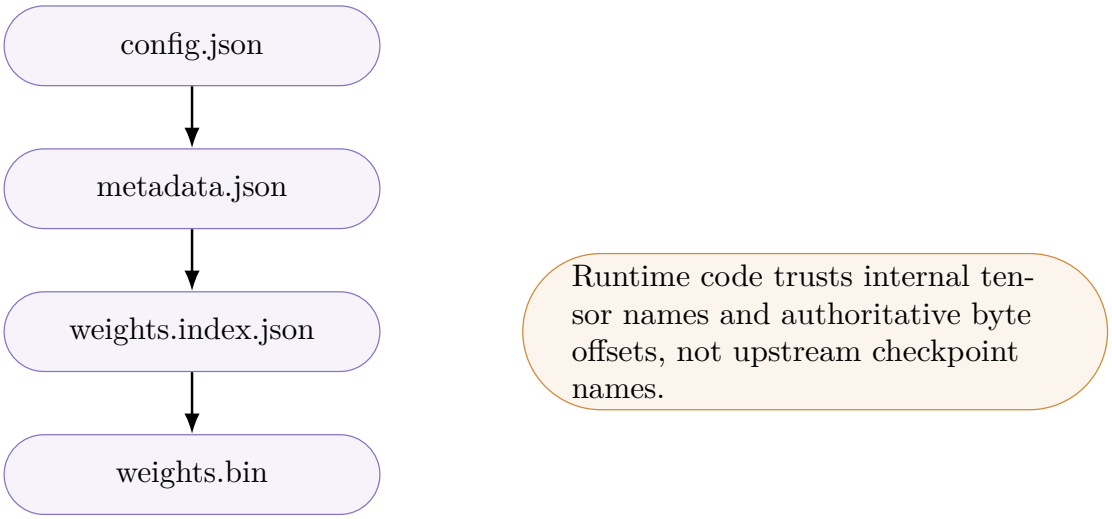

Figure 14.1: Internal artifact format and its authority boundaries.

```
{
  "blocks.0.attn.qkv.weight": {
    "dtype": "float32",
    "shape": [2304, 768],
    "offset": 1048576,
    "nbytes": 7077888,
    "source_name": "transformer.h.0.attn.c_attn.weight"
  }
}
```

Listing 14.1: Schematic weights.index.json entry.

Intuition

Direct loading is convenient; conversion is the contract boundary.

Mathematical or contract statement

A valid artifact has `config.json`, `metadata.json`, `weights.index.json`, and `weights.bin`. Index offsets and byte counts are authoritative.

Implementation interpretation

Implement `SafeTensorLoader` for direct reads, `ArtifactConverter` for mapping into internal names, and `WeightLoader` for runtime artifacts.

How to verify this works

- Check shapes and dtypes at the boundary.
- Compare deterministic outputs against PyTorch golden vectors.
- Record tolerance, seed, model artifact, and backend.
- Do not benchmark until validation passes.

Common failure modes

- Letting framework defaults choose dtype or layout.
- Testing only a batch size of one.
- Depending on upstream checkpoint names in runtime code.
- Treating benchmark output as validation.

Exercises

E14.1. State the central invariant in one sentence.
Hint. Use the conceptual guide vocabulary.

E14.2. Construct a toy example with small shapes.
Hint. Use $B = 2$, $T = 3$, $H = 8$ where possible.

E14.3. Name a validation test that would catch a plausible bug.
Hint. Prefer generated golden vectors to handwritten long arrays.

Chapter 15

Tokenizers and embedders

Tokenizers convert text to ids and ids back to text. Tokenizer validation is separate from model-math validation.

Intuition

The model consumes token ids, not strings.

Mathematical or contract statement

Token embeddings map $I \in \mathbb{N}^{B \times T}$ to $X \in \mathbb{R}^{B \times T \times H}$ by table lookup; learned positions add a row per position.

Implementation interpretation

For correctness vectors, `input_ids` are authoritative and prompt text is optional metadata.

How to verify this works

- Check shapes and dtypes at the boundary.
- Compare deterministic outputs against PyTorch golden vectors.
- Record tolerance, seed, model artifact, and backend.
- Do not benchmark until validation passes.

Common failure modes

- Letting framework defaults choose dtype or layout.
- Testing only a batch size of one.
- Depending on upstream checkpoint names in runtime code.
- Treating benchmark output as validation.

Exercises

E15.1. State the central invariant in one sentence.
Hint. Use the conceptual guide vocabulary.

E15.2. Construct a toy example with small shapes.
Hint. Use $B = 2$, $T = 3$, $H = 8$ where possible.

E15.3. Name a validation test that would catch a plausible bug.
Hint. Prefer generated golden vectors to handwritten long arrays.

Chapter 16

Decoder-only model architecture

The engine targets decoder-only autoregressive models. GPT-2 proves the baseline dense decoder stack.

Intuition

A decoder sees past tokens and never future tokens.

Mathematical or contract statement

The model maps ids to logits $Z \in \mathbb{R}^{B \times T \times V}$ through embeddings, blocks, final normalisation, and an LM head.

Implementation interpretation

Keep block variants behind configuration and model-family adapters.

How to verify this works

- Check shapes and dtypes at the boundary.
- Compare deterministic outputs against PyTorch golden vectors.
- Record tolerance, seed, model artifact, and backend.
- Do not benchmark until validation passes.

Common failure modes

- Letting framework defaults choose dtype or layout.
- Testing only a batch size of one.
- Depending on upstream checkpoint names in runtime code.
- Treating benchmark output as validation.

Exercises

E16.1. State the central invariant in one sentence.
Hint. Use the conceptual guide vocabulary.

E16.2. Construct a toy example with small shapes.
Hint. Use $B = 2$, $T = 3$, $H = 8$ where possible.

E16.3. Name a validation test that would catch a plausible bug.
Hint. Prefer generated golden vectors to handwritten long arrays.

Chapter 17

Prefill, decode, and generation loops

Prefill processes a prompt and builds cache state. Decode processes incremental tokens using cached state.

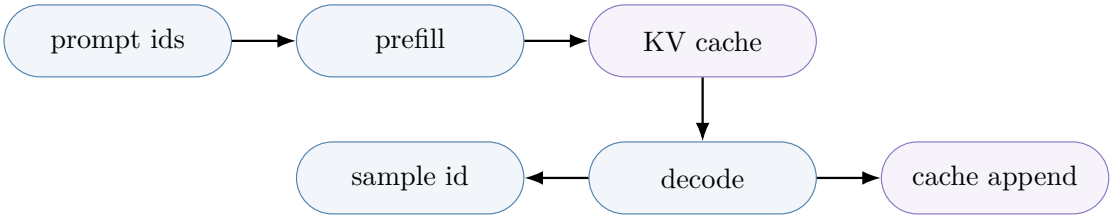

Figure 17.1: Prefill builds cache state; decode reuses and appends it.

Intuition

Generation is stateful and must be tested as stateful.

Mathematical or contract statement

Cache equivalence states that full recomputation on prompt plus token equals prefill on the prompt followed by cached decode within tolerance.

Implementation interpretation

Expose `Prefill`, `Decode`, and `GenerationLoop`. Keep stop conditions and sampler policy explicit.

How to verify this works

- Check shapes and dtypes at the boundary.
- Compare deterministic outputs against PyTorch golden vectors.
- Record tolerance, seed, model artifact, and backend.
- Do not benchmark until validation passes.

Common failure modes

- Letting framework defaults choose dtype or layout.
- Testing only a batch size of one.
- Depending on upstream checkpoint names in runtime code.
- Treating benchmark output as validation.

Exercises

E17.1. State the central invariant in one sentence.
Hint. Use the conceptual guide vocabulary.

E17.2. Construct a toy example with small shapes.
Hint. Use $B = 2$, $T = 3$, $H = 8$ where possible.

E17.3. Name a validation test that would catch a plausible bug.
Hint. Prefer generated golden vectors to handwritten long arrays.

Chapter 18

KV-cache abstraction and cache policies

The KV cache stores recurrent decode state per layer. Phase 0 uses append-only cache; later phases add sliding-window policies.

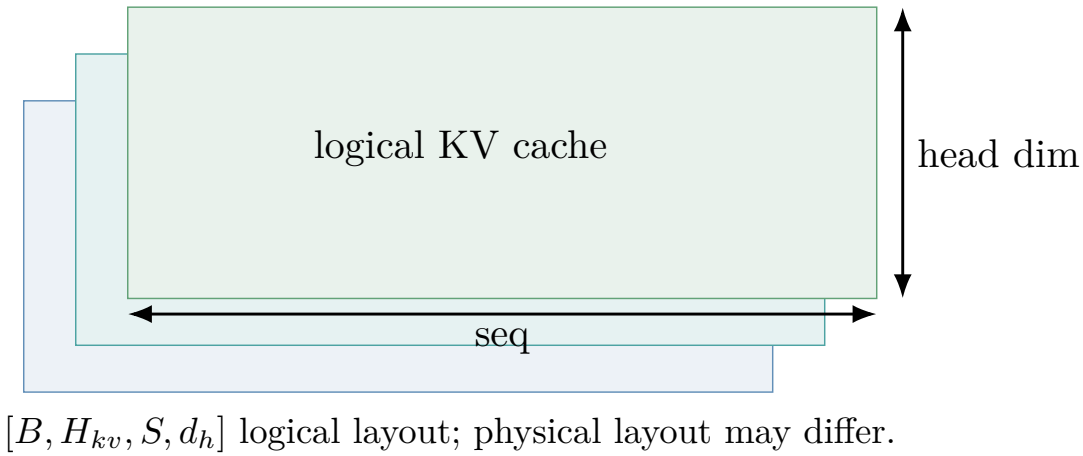

Figure 18.1: Canonical KV-cache logical layout.

Intuition

A cache is a visibility rule plus stored tensors.

Mathematical or contract statement

For each layer ℓ, keys and values have logical shape $K_\ell, V_\ell \in \mathbb{R}^{B \times H_{kv} \times S \times d_h}$. In phase 0, the visible span for decode step t is the full accumulated prefix $s \leq S + t$; sliding-window policies later replace this with a bounded visibility predicate.

Implementation interpretation

Implement `KVCache`, `CachePolicy`, `AppendOnlyCache`, and `SlidingWindowCache`; only append-only is active in phase 0.

How to verify this works

- Check shapes and dtypes at the boundary.
- Compare deterministic outputs against PyTorch golden vectors.
- Record tolerance, seed, model artifact, and backend.
- Do not benchmark until validation passes.

Common failure modes

- Letting framework defaults choose dtype or layout.
- Testing only a batch size of one.
- Depending on upstream checkpoint names in runtime code.
- Treating benchmark output as validation.

Exercises

E18.1. State the central invariant in one sentence.
Hint. Use the conceptual guide vocabulary.

E18.2. Construct a toy example with small shapes.
Hint. Use $B = 2$, $T = 3$, $H = 8$ where possible.

E18.3. Name a validation test that would catch a plausible bug.
Hint. Prefer generated golden vectors to handwritten long arrays.

Chapter 19

CLI, validation, and benchmarking contracts

The CLI contract makes all runtimes comparable from automation. Required commands are run, generate, test, and bench.

```
engine run --model <model_dir> --input-ids 15496,995
engine generate --model <model_dir> --input-ids 15496,995 --max-new-tokens 16
engine test --model <model_dir> --test-vector <path>
engine bench --model <model_dir> --suite <suite_config>
```

Listing 19.1: Common CLI operations.

Intuition

A CLI is a reproducibility surface.

Mathematical or contract statement

Automation-facing commands must support JSON output and nonzero exit codes for failure. Correctness workflows should prefer input ids.

Implementation interpretation

The model path points to a converted artifact directory, never an upstream checkpoint cache.

How to verify this works

- Check shapes and dtypes at the boundary.
- Compare deterministic outputs against PyTorch golden vectors.
- Record tolerance, seed, model artifact, and backend.
- Do not benchmark until validation passes.

Common failure modes

- Letting framework defaults choose dtype or layout.
- Testing only a batch size of one.
- Depending on upstream checkpoint names in runtime code.
- Treating benchmark output as validation.

Exercises

E19.1. State the central invariant in one sentence.
Hint. Use the conceptual guide vocabulary.

E19.2. Construct a toy example with small shapes.
Hint. Use $B = 2$, $T = 3$, $H = 8$ where possible.

E19.3. Name a validation test that would catch a plausible bug.
Hint. Prefer generated golden vectors to handwritten long arrays.

Chapter 20

High-level PyTorch/Rust/C++ interface design

The same conceptual interfaces appear in PyTorch, C++, and Rust even when language idioms differ.

Table 20.1: Abstract components required by the conceptual guide.

Component	Responsibility
`ModelConfig`	Validated engine-native model configuration.
`WeightLoader`, `SafeTensorLoader`, `ArtifactConverter`	Direct SafeTensors reading where useful and conversion into internal artifacts.
`Tensor`, `Device`, `DType`	Shape, dtype, layout, storage, and placement.
`Tokenizer`, `Embedder`, `TokenEmbedding`	Text boundary and embedding lookup.
`PositionalEncoding`, `LearnedAbsolutePositionEncoding`, `RoPE`	Position strategies.
`Attention`, `SelfAttention`, `MultiHeadAttention`, `MQA`, `GQA`, `AttentionKernel`	Attention semantics and backend kernels.
`MLPBlock`, `GELUFFN`, `SwiGLU`, `GeGLU`, `Activation`	Feed-forward families and activations.
`LayerNorm`, `RMSNorm`, `ResidualBlock`, `TransformerBlock`	Normalisation and block composition.
`MambaBlock`	Hybrid or recurrent block abstraction.
`KVCache`, `CachePolicy`, `AppendOnlyCache`, `SlidingWindowCache`	Cache storage and visibility policy.
`Sampler`, `GreedySampler`, `TemperatureSampler`, `TopKSampler`, `TopPSampler`	Logit-to-token policies.
`GenerationLoop`, `Prefill`, `Decode`	Autoregressive scheduling.
`Validator`, `GoldenVectorExporter`, `BenchmarkRunner`, `Profiler`	Correctness, reproducibility, measurement, and diagnostics.

Intuition

Interfaces should expose invariants, not implementation accidents.

Mathematical or contract statement

The engine vocabulary includes config, loading, tensors, devices, tokenizers, modules, caches, samplers, generation, validation, benchmarking, and profiling.

Implementation interpretation

Use PyTorch for clarity, C++ for portable systems performance, Rust for ownership discipline, and CUDA as a later backend.

How to verify this works

- Check shapes and dtypes at the boundary.
- Compare deterministic outputs against PyTorch golden vectors.
- Record tolerance, seed, model artifact, and backend.
- Do not benchmark until validation passes.

Common failure modes

- Letting framework defaults choose dtype or layout.
- Testing only a batch size of one.
- Depending on upstream checkpoint names in runtime code.
- Treating benchmark output as validation.

Exercises

E20.1. State the central invariant in one sentence.
Hint. Use the conceptual guide vocabulary.

E20.2. Construct a toy example with small shapes.
Hint. Use $B = 2$, $T = 3$, $H = 8$ where possible.

E20.3. Name a validation test that would catch a plausible bug.
Hint. Prefer generated golden vectors to handwritten long arrays.

Part III

Phase 0: GPT-2 CPU-first correctness

Chapter 21

GPT-2 architecture

GPT-2 is the phase-0 architecture because it is simple enough to finish and complex enough to exercise the decoder runtime. [Radford et al., 2019]

Module	Phase-0 form	Contract point
Embeddings	token plus learned positions	No RoPE in GPT-2.
Attention	causal MHA	KV heads equal query heads.
MLP	GELU feed-forward	Biases present.
Cache	append-only	Full accumulated span visible.
Validation	PyTorch oracle	C++ and Rust compare within tolerance.

Table 21.1: Phase-0 GPT-2 scope.

Intuition

The baseline proves the whole loop before newer variants arrive.

Mathematical or contract statement

A GPT-2 block is pre-LayerNorm causal MHA plus GELU MLP with residual connections.

Implementation interpretation

Implement only the GPT-2 CPU float32 correctness path in phase 0.

How to verify this works

- Check shapes and dtypes at the boundary.
- Compare deterministic outputs against PyTorch golden vectors.
- Record tolerance, seed, model artifact, and backend.
- Do not benchmark until validation passes.

Common failure modes

- Letting framework defaults choose dtype or layout.
- Testing only a batch size of one.
- Depending on upstream checkpoint names in runtime code.
- Treating benchmark output as validation.

Exercises

E21.1. State the central invariant in one sentence.
Hint. Use the conceptual guide vocabulary.

E21.2. Construct a toy example with small shapes.
Hint. Use $B = 2$, $T = 3$, $H = 8$ where possible.

E21.3. Name a validation test that would catch a plausible bug.
Hint. Prefer generated golden vectors to handwritten long arrays.

Chapter 22

Token and learned position embeddings

GPT-2 uses token embeddings and learned absolute position embeddings.

Intuition

The first hidden tensor is where token identity meets position.

Mathematical or contract statement

Let $E^{tok} \in \mathbb{R}^{V \times H}$ be the token table and $E^{pos} \in \mathbb{R}^{P \times H}$ be the learned position table. If c_b is the cache length before the current segment, then $p_{b,t} = c_b + t$ and

$$X_{b,t,:} = E^{tok}_{I_{b,t},:} + E^{pos}_{p_{b,t},:}, \qquad 0 \le p_{b,t} < P.$$

Implementation interpretation

During decode, compute the position id from cache length; do not restart at zero.

How to verify this works

- Check shapes and dtypes at the boundary.
- Compare deterministic outputs against PyTorch golden vectors.
- Record tolerance, seed, model artifact, and backend.
- Do not benchmark until validation passes.

Common failure modes

- Letting framework defaults choose dtype or layout.
- Testing only a batch size of one.
- Depending on upstream checkpoint names in runtime code.
- Treating benchmark output as validation.

Exercises

E22.1. State the central invariant in one sentence.
Hint. Use the conceptual guide vocabulary.

E22.2. Construct a toy example with small shapes.
Hint. Use $B = 2$, $T = 3$, $H = 8$ where possible.

E22.3. Name a validation test that would catch a plausible bug.
Hint. Prefer generated golden vectors to handwritten long arrays.

Chapter 23

LayerNorm

LayerNorm normalises each token's hidden vector and applies learned scale and bias.

$$\mu = \frac{1}{H}\sum_{j=1}^{H} x_j, \quad \sigma^2 = \frac{1}{H}\sum_{j=1}^{H}(x_j - \mu)^2, \quad \mathrm{LN}(x)_j = \gamma_j \frac{x_j - \mu}{\sqrt{\sigma^2 + \epsilon}} + \beta_j.$$

Intuition

LayerNorm is a per-token reduction over hidden channels.

Mathematical or contract statement

For each token vector $x \in \mathbb{R}^H$, the displayed formula is the phase-0 normalisation contract. The reduction axis is the hidden dimension only.

Implementation interpretation

Use config epsilon and float32 accumulation for the CPU reference.

How to verify this works

- Check shapes and dtypes at the boundary.
- Compare deterministic outputs against PyTorch golden vectors.
- Record tolerance, seed, model artifact, and backend.
- Do not benchmark until validation passes.

Common failure modes

- Letting framework defaults choose dtype or layout.
- Testing only a batch size of one.
- Depending on upstream checkpoint names in runtime code.
- Treating benchmark output as validation.

Exercises

E23.1. State the central invariant in one sentence.
Hint. Use the conceptual guide vocabulary.

E23.2. Construct a toy example with small shapes.
Hint. Use $B = 2$, $T = 3$, $H = 8$ where possible.

E23.3. Name a validation test that would catch a plausible bug.
Hint. Prefer generated golden vectors to handwritten long arrays.

Chapter 24

Causal self-attention

Causal self-attention mixes visible previous tokens while hiding the future.

$$\alpha_{b,h,t,s} = \mathrm{softmax}_s\left(\frac{\langle Q_{b,h,t,:}, K_{b,h,s,:}\rangle}{\sqrt{d_h}} + M_{t,s}\right), \qquad O_{b,h,t,:} = \sum_{s=0}^{S-1} \alpha_{b,h,t,s} V_{b,h,s,:}.$$

For phase-0 GPT-2, $H_{kv} = H_q$, so query head h reads key/value head h. The causal mask $M_{t,s}$ is 0 for visible key positions and $-\infty$ for future positions.

Intuition

Attention is content-addressed lookup under a no-future rule.

Mathematical or contract statement

Scaled dot-product attention applies the causal mask before softmax. Masking after softmax is a different operation and generally changes probabilities.

Implementation interpretation

Split fused QKV projection into query, key, and value segments, then reshape by head.

How to verify this works

- Check shapes and dtypes at the boundary.
- Compare deterministic outputs against PyTorch golden vectors.
- Record tolerance, seed, model artifact, and backend.
- Do not benchmark until validation passes.

Common failure modes

- Letting framework defaults choose dtype or layout.
- Testing only a batch size of one.
- Depending on upstream checkpoint names in runtime code.
- Treating benchmark output as validation.

Exercises

E24.1. State the central invariant in one sentence.
Hint. Use the conceptual guide vocabulary.

E24.2. Construct a toy example with small shapes.

Hint. Use $B = 2$, $T = 3$, $H = 8$ where possible.

E24.3. Name a validation test that would catch a plausible bug.

Hint. Prefer generated golden vectors to handwritten long arrays.

Chapter 25

GELU MLP

The GPT-2 MLP expands hidden states, applies GELU, then projects back.

$$\mathrm{GELU}(x) \approx \frac{x}{2}\left(1 + \tanh\left(\sqrt{\frac{2}{\pi}}\left(x + 0.044715x^3\right)\right)\right).$$

Intuition

The MLP gives each token nonlinear channel mixing.

Mathematical or contract statement

For a token vector $x \in \mathbb{R}^H$, $W_1 \in \mathbb{R}^{F\times H}$, $b_1 \in \mathbb{R}^F$, $W_2 \in \mathbb{R}^{H\times F}$, and $b_2 \in \mathbb{R}^H$,

$$y = \mathrm{GELU}(xW_1^\top + b_1)W_2^\top + b_2 \in \mathbb{R}^H.$$

Implementation interpretation

Use the same GELU approximation as the PyTorch reference.

How to verify this works

- Check shapes and dtypes at the boundary.
- Compare deterministic outputs against PyTorch golden vectors.
- Record tolerance, seed, model artifact, and backend.
- Do not benchmark until validation passes.

Common failure modes

- Letting framework defaults choose dtype or layout.
- Testing only a batch size of one.
- Depending on upstream checkpoint names in runtime code.
- Treating benchmark output as validation.

Exercises

E25.1. State the central invariant in one sentence.
Hint. Use the conceptual guide vocabulary.

E25.2. Construct a toy example with small shapes.

Hint. Use $B = 2$, $T = 3$, $H = 8$ where possible.

E25.3. Name a validation test that would catch a plausible bug.

Hint. Prefer generated golden vectors to handwritten long arrays.

Chapter 26

Transformer block composition

A GPT-2 block composes LayerNorm, attention, residual addition, LayerNorm, MLP, and residual addition.

Intuition

Residuals preserve an identity path through every block.

Mathematical or contract statement

For block ℓ, pre-norm form is

$$U^{(\ell)} = X^{(\ell)} + \mathrm{Attn}_\ell(\mathrm{LN}_{1,\ell}(X^{(\ell)})), \qquad X^{(\ell+1)} = U^{(\ell)} + \mathrm{MLP}_\ell(\mathrm{LN}_{2,\ell}(U^{(\ell)})).$$

Implementation interpretation

Export block-boundary tensors so mismatches localise quickly.

How to verify this works

- Check shapes and dtypes at the boundary.
- Compare deterministic outputs against PyTorch golden vectors.
- Record tolerance, seed, model artifact, and backend.
- Do not benchmark until validation passes.

Common failure modes

- Letting framework defaults choose dtype or layout.
- Testing only a batch size of one.
- Depending on upstream checkpoint names in runtime code.
- Treating benchmark output as validation.

Exercises

E26.1. State the central invariant in one sentence.
Hint. Use the conceptual guide vocabulary.

E26.2. Construct a toy example with small shapes.
Hint. Use $B = 2$, $T = 3$, $H = 8$ where possible.

E26.3. Name a validation test that would catch a plausible bug.
Hint. Prefer generated golden vectors to handwritten long arrays.

Chapter 27

Final normalisation and language-modelling head

The final LayerNorm and language-modelling head produce vocabulary logits.

Intuition

The head turns hidden vectors into token scores.

Mathematical or contract statement

If $X \in \mathbb{R}^{B \times T \times H}$ and $W_{lm} \in \mathbb{R}^{V \times H}$, logits satisfy $Z_{b,t,:} = X_{b,t,:} W_{lm}^{\top} \in \mathbb{R}^{V}$. GPT-2 uses tied token embeddings in many checkpoints, but the internal artifact exposes the LM head explicitly.

Implementation interpretation

Load an explicit `lm_head.weight` even when embeddings are tied.

How to verify this works

- Check shapes and dtypes at the boundary.
- Compare deterministic outputs against PyTorch golden vectors.
- Record tolerance, seed, model artifact, and backend.
- Do not benchmark until validation passes.

Common failure modes

- Letting framework defaults choose dtype or layout.
- Testing only a batch size of one.
- Depending on upstream checkpoint names in runtime code.
- Treating benchmark output as validation.

Exercises

E27.1. State the central invariant in one sentence.
Hint. Use the conceptual guide vocabulary.

E27.2. Construct a toy example with small shapes.
Hint. Use $B = 2$, $T = 3$, $H = 8$ where possible.

E27.3. Name a validation test that would catch a plausible bug.
Hint. Prefer generated golden vectors to handwritten long arrays.

Chapter 28

PyTorch reference engine

The PyTorch reference is the phase-0 oracle and vector exporter. [PyTorch Contributors, 2026]

```python
def forward(input_ids, cache=None):
    position_ids = make_position_ids(input_ids, cache)
    x = token_embedding(input_ids) + position_embedding(position_ids)
    for block in blocks:
        x = block(x, cache=cache)
    x = final_layer_norm(x)
    return x @ lm_head_weight.T
```

Listing 28.1: Reference-forward sketch.

Intuition

The reference should be boring, explicit, and inspectable.

Mathematical or contract statement

PyTorch defines the expected f_θ over converted artifacts.

Implementation interpretation

Construct the model from internal artifacts, not from an upstream pretrained class.

How to verify this works

- Check shapes and dtypes at the boundary.
- Compare deterministic outputs against PyTorch golden vectors.
- Record tolerance, seed, model artifact, and backend.
- Do not benchmark until validation passes.

Common failure modes

- Letting framework defaults choose dtype or layout.
- Testing only a batch size of one.
- Depending on upstream checkpoint names in runtime code.
- Treating benchmark output as validation.

Exercises

E28.1. State the central invariant in one sentence.
Hint. Use the conceptual guide vocabulary.

E28.2. Construct a toy example with small shapes.
Hint. Use $B = 2$, $T = 3$, $H = 8$ where possible.

E28.3. Name a validation test that would catch a plausible bug.
Hint. Prefer generated golden vectors to handwritten long arrays.

Chapter 29

Exporting golden vectors

Golden vectors are generated records used by all implementations.

```
{
  "name": "smoke_hello",
  "model_family": "gpt2",
  "model_name": "gpt2",
  "input_ids": [15496, 995],
  "dtype": "float32",
  "tolerance": {"atol": 1e-4, "rtol": 1e-4}
}
```

Listing 29.1: Golden-vector sidecar sketch.

Intuition

A vector is a reproducible experiment, not a magic constant.

Mathematical or contract statement

JSON carries metadata and NPZ carries larger arrays with a matching sidecar.

Implementation interpretation

Do not hard-code long vectors in LaTeX or source comments.

How to verify this works

- Check shapes and dtypes at the boundary.
- Compare deterministic outputs against PyTorch golden vectors.
- Record tolerance, seed, model artifact, and backend.
- Do not benchmark until validation passes.

Common failure modes

- Letting framework defaults choose dtype or layout.
- Testing only a batch size of one.
- Depending on upstream checkpoint names in runtime code.
- Treating benchmark output as validation.

Exercises

E29.1. State the central invariant in one sentence.
Hint. Use the conceptual guide vocabulary.

E29.2. Construct a toy example with small shapes.
Hint. Use $B = 2$, $T = 3$, $H = 8$ where possible.

E29.3. Name a validation test that would catch a plausible bug.
Hint. Prefer generated golden vectors to handwritten long arrays.

Chapter 30

C++ correctness engine

The C++ correctness engine is CPU float32 and consumes the same artifacts as PyTorch. [International Organization for Standardization, 2020, Kitware, 2026]

```
struct TensorView {
    DType dtype;
    std::vector<std::size_t> shape;
    std::span<const std::byte> bytes;
};
```

Listing 30.1: C++ artifact view sketch.

Intuition

C++ earns trust by explicit memory ownership and checked views.

Mathematical or contract statement

The C++ engine implements config, loader, tensor views, CPU ops, modules, cache, validation, and CLI.

Implementation interpretation

Add optimisation only after scalar correctness passes.

How to verify this works

- Check shapes and dtypes at the boundary.
- Compare deterministic outputs against PyTorch golden vectors.
- Record tolerance, seed, model artifact, and backend.
- Do not benchmark until validation passes.

Common failure modes

- Letting framework defaults choose dtype or layout.
- Testing only a batch size of one.
- Depending on upstream checkpoint names in runtime code.
- Treating benchmark output as validation.

Exercises

E30.1. State the central invariant in one sentence.
Hint. Use the conceptual guide vocabulary.

E30.2. Construct a toy example with small shapes.
Hint. Use $B = 2$, $T = 3$, $H = 8$ where possible.

E30.3. Name a validation test that would catch a plausible bug.
Hint. Prefer generated golden vectors to handwritten long arrays.

Chapter 31

Rust correctness engine

The Rust correctness engine mirrors the C++ semantics while using ownership and typed errors. [Klabnik et al., 2026, The Rust Project Developers, 2025]

```rust
pub trait WeightLoader {
    fn tensor<'a>(&'a self, name: &str) -> Result<Tensor<'a>, Error>;
}
```

Listing 31.1: Rust loader trait sketch.

Intuition

Rust helps prevent stale or aliased cache and tensor views.

Mathematical or contract statement

Rust consumes the same artifacts and compares against the same vectors.

Implementation interpretation

Use `Result` for malformed artifacts and validation failures.

How to verify this works

- Check shapes and dtypes at the boundary.
- Compare deterministic outputs against PyTorch golden vectors.
- Record tolerance, seed, model artifact, and backend.
- Do not benchmark until validation passes.

Common failure modes

- Letting framework defaults choose dtype or layout.
- Testing only a batch size of one.
- Depending on upstream checkpoint names in runtime code.
- Treating benchmark output as validation.

Exercises

E31.1. State the central invariant in one sentence.
Hint. Use the conceptual guide vocabulary.

E31.2. Construct a toy example with small shapes.

Hint. Use $B = 2$, $T = 3$, $H = 8$ where possible.

E31.3. Name a validation test that would catch a plausible bug.

Hint. Prefer generated golden vectors to handwritten long arrays.

Chapter 32

Cross-language validation

Cross-language validation compares PyTorch, C++, and Rust over one artifact directory.

Intuition

Three runtimes are valuable only if disagreement is visible.

Mathematical or contract statement

Shapes, dtype, hidden states, logits, greedy ids, generation, and cache equivalence are required checks.

Implementation interpretation

Build one comparison policy and use it for every implementation.

How to verify this works

- Check shapes and dtypes at the boundary.
- Compare deterministic outputs against PyTorch golden vectors.
- Record tolerance, seed, model artifact, and backend.
- Do not benchmark until validation passes.

Common failure modes

- Letting framework defaults choose dtype or layout.
- Testing only a batch size of one.
- Depending on upstream checkpoint names in runtime code.
- Treating benchmark output as validation.

Exercises

E32.1. State the central invariant in one sentence.
Hint. Use the conceptual guide vocabulary.

E32.2. Construct a toy example with small shapes.
Hint. Use $B = 2$, $T = 3$, $H = 8$ where possible.

E32.3. Name a validation test that would catch a plausible bug.
Hint. Prefer generated golden vectors to handwritten long arrays.

Chapter 33

Cache-equivalence testing

Cache equivalence is mandatory in phase 0.

Intuition

The cache is correct only if removing it does not change the answer.

Mathematical or contract statement

For a prompt $u_{1:m}$ and next token v, the final-position logits from full recomputation on $(u_{1:m}, v)$ must match the logits from prefill on $u_{1:m}$ followed by cached decode on v, within tolerance. The comparison also checks that every layer's cache length increases from m to $m + 1$.

Implementation interpretation

Test final-position logits, cache lengths, and per-layer append behaviour.

How to verify this works

- Check shapes and dtypes at the boundary.
- Compare deterministic outputs against PyTorch golden vectors.
- Record tolerance, seed, model artifact, and backend.
- Do not benchmark until validation passes.

Common failure modes

- Letting framework defaults choose dtype or layout.
- Testing only a batch size of one.
- Depending on upstream checkpoint names in runtime code.
- Treating benchmark output as validation.

Exercises

E33.1. State the central invariant in one sentence.
Hint. Use the conceptual guide vocabulary.

E33.2. Construct a toy example with small shapes.
Hint. Use $B = 2$, $T = 3$, $H = 8$ where possible.

E33.3. Name a validation test that would catch a plausible bug.
Hint. Prefer generated golden vectors to handwritten long arrays.

Chapter 34

Baseline benchmarking

Benchmarking comes after correctness and measures prefill latency, decode latency, end-to-end generation, and throughput.

Intuition

A benchmark is a measurement of a known-correct path.

Mathematical or contract statement

Useful metrics include prompt length, generated tokens, batch size, latency, tokens per second, and cache bytes.

Implementation interpretation

Store benchmark suites separately from golden vectors.

How to verify this works

- Check shapes and dtypes at the boundary.
- Compare deterministic outputs against PyTorch golden vectors.
- Record tolerance, seed, model artifact, and backend.
- Do not benchmark until validation passes.

Common failure modes

- Letting framework defaults choose dtype or layout.
- Testing only a batch size of one.
- Depending on upstream checkpoint names in runtime code.
- Treating benchmark output as validation.

Exercises

E34.1. State the central invariant in one sentence.
Hint. Use the conceptual guide vocabulary.

E34.2. Construct a toy example with small shapes.
Hint. Use $B = 2$, $T = 3$, $H = 8$ where possible.

E34.3. Name a validation test that would catch a plausible bug.
Hint. Prefer generated golden vectors to handwritten long arrays.

Part IV

Modern transformer phases

Chapter 35

LLaMA-style architectural changes

After GPT-2 correctness, the roadmap adds LLaMA-style changes such as RoPE, RMSNorm, and gated MLPs. [Touvron et al., 2023]

Intuition

Modernisation changes contracts, not just modules.

Mathematical or contract statement

A modern dense decoder remains autoregressive but may change positions, norms, MLPs, and tensor names.

Implementation interpretation

Extend config, converter, validator, and tests before adding kernels.

How to verify this works

- Check shapes and dtypes at the boundary.
- Compare deterministic outputs against PyTorch golden vectors.
- Record tolerance, seed, model artifact, and backend.
- Do not benchmark until validation passes.

Common failure modes

- Letting framework defaults choose dtype or layout.
- Testing only a batch size of one.
- Depending on upstream checkpoint names in runtime code.
- Treating benchmark output as validation.

Exercises

E35.1. State the central invariant in one sentence.
Hint. Use the conceptual guide vocabulary.

E35.2. Construct a toy example with small shapes.
Hint. Use $B = 2$, $T = 3$, $H = 8$ where possible.

E35.3. Name a validation test that would catch a plausible bug.
Hint. Prefer generated golden vectors to handwritten long arrays.

Chapter 36

RoPE

RoPE rotates query and key channel pairs by position and is not enabled for phase-0 GPT-2. [Su et al., 2021]

$$\theta_{p,j} = p\,\omega_j, \qquad \omega_j = \theta_{\text{base}}^{-2j/d_h},$$

and each even/odd channel pair is rotated as

$$\begin{pmatrix} x'_{2j} \\ x'_{2j+1} \end{pmatrix} = \begin{pmatrix} \cos\theta_{p,j} & -\sin\theta_{p,j} \\ \sin\theta_{p,j} & \cos\theta_{p,j} \end{pmatrix} \begin{pmatrix} x_{2j} \\ x_{2j+1} \end{pmatrix}.$$

Intuition

RoPE moves position information into attention geometry.

Mathematical or contract statement

RoPE applies a block-diagonal rotation to Q and K, not V. Decode must use absolute position ids consistent with prefill so cached rotated keys match full recomputation.

Implementation interpretation

Store cached keys after the same rotation used in full recomputation.

How to verify this works

- Check shapes and dtypes at the boundary.
- Compare deterministic outputs against PyTorch golden vectors.
- Record tolerance, seed, model artifact, and backend.
- Do not benchmark until validation passes.

Common failure modes

- Letting framework defaults choose dtype or layout.
- Testing only a batch size of one.
- Depending on upstream checkpoint names in runtime code.
- Treating benchmark output as validation.

Exercises

E36.1. State the central invariant in one sentence.
Hint. Use the conceptual guide vocabulary.

E36.2. Construct a toy example with small shapes.
Hint. Use $B = 2$, $T = 3$, $H = 8$ where possible.

E36.3. Name a validation test that would catch a plausible bug.
Hint. Prefer generated golden vectors to handwritten long arrays.

Chapter 37

RMSNorm

RMSNorm is common in newer decoder families and normalises by root mean square. [Zhang and Sennrich, 2019]

$$\mathrm{RMSNorm}(x)_j = \gamma_j \frac{x_j}{\sqrt{H^{-1} \sum_{k=1}^{H} x_k^2 + \epsilon}}.$$

Intuition

RMSNorm scales without mean subtraction.

Mathematical or contract statement

RMSNorm divides by the square root of the mean square and multiplies by a learned scale.

Implementation interpretation

Implement it as a separate module from LayerNorm.

How to verify this works

- Check shapes and dtypes at the boundary.
- Compare deterministic outputs against PyTorch golden vectors.
- Record tolerance, seed, model artifact, and backend.
- Do not benchmark until validation passes.

Common failure modes

- Letting framework defaults choose dtype or layout.
- Testing only a batch size of one.
- Depending on upstream checkpoint names in runtime code.
- Treating benchmark output as validation.

Exercises

E37.1. State the central invariant in one sentence.
Hint. Use the conceptual guide vocabulary.

E37.2. Construct a toy example with small shapes.
Hint. Use $B = 2$, $T = 3$, $H = 8$ where possible.

E37.3. Name a validation test that would catch a plausible bug.

Hint. Prefer generated golden vectors to handwritten long arrays.

Chapter 38

Gated MLPs: SwiGLU and GeGLU

Gated MLPs use a learned gate and are common in modern decoder families. [Shazeer, 2020]

$$\mathrm{SwiGLU}(x) = \left(\mathrm{SiLU}(xW_g^\top + b_g) \odot (xW_u^\top + b_u)\right) W_d^\top + b_d,$$

where $W_g, W_u \in \mathbb{R}^{F\times H}$, $W_d \in \mathbb{R}^{H\times F}$, and biases are present only for model families whose config enables them.

Intuition

The gate controls which expanded channels flow.

Mathematical or contract statement

SwiGLU and GeGLU compute two expanded vectors in $\mathbb{R}^F$, combine them with an elementwise product, and then apply the down projection back to $\mathbb{R}^H$.

Implementation interpretation

Map gate, up, and down tensors separately in the converter.

How to verify this works

- Check shapes and dtypes at the boundary.
- Compare deterministic outputs against PyTorch golden vectors.
- Record tolerance, seed, model artifact, and backend.
- Do not benchmark until validation passes.

Common failure modes

- Letting framework defaults choose dtype or layout.
- Testing only a batch size of one.
- Depending on upstream checkpoint names in runtime code.
- Treating benchmark output as validation.

Exercises

E38.1. State the central invariant in one sentence.
Hint. Use the conceptual guide vocabulary.

E38.2. Construct a toy example with small shapes.

Hint. Use $B = 2$, $T = 3$, $H = 8$ where possible.

E38.3. Name a validation test that would catch a plausible bug.

Hint. Prefer generated golden vectors to handwritten long arrays.

Chapter 39

MQA

MQA uses many query heads but one or few key/value heads, reducing decode cache bandwidth. [Shazeer, 2019]

Intuition

Fewer KV heads mean fewer cache bytes per token.

Mathematical or contract statement

Strict MQA is the special case $H_{kv} = 1$ while H_q may be larger. Implementations can share the same grouped-head machinery as GQA, but the config must still record the exact number of key/value heads.

Implementation interpretation

Generalise attention code without changing phase-0 GPT-2 equality.

How to verify this works

- Check shapes and dtypes at the boundary.
- Compare deterministic outputs against PyTorch golden vectors.
- Record tolerance, seed, model artifact, and backend.
- Do not benchmark until validation passes.

Common failure modes

- Letting framework defaults choose dtype or layout.
- Testing only a batch size of one.
- Depending on upstream checkpoint names in runtime code.
- Treating benchmark output as validation.

Exercises

E39.1. State the central invariant in one sentence.
Hint. Use the conceptual guide vocabulary.

E39.2. Construct a toy example with small shapes.
Hint. Use $B = 2$, $T = 3$, $H = 8$ where possible.

E39.3. Name a validation test that would catch a plausible bug.
Hint. Prefer generated golden vectors to handwritten long arrays.

Chapter 40

GQA

GQA groups query heads over fewer key/value heads. [Ainslie et al., 2023]

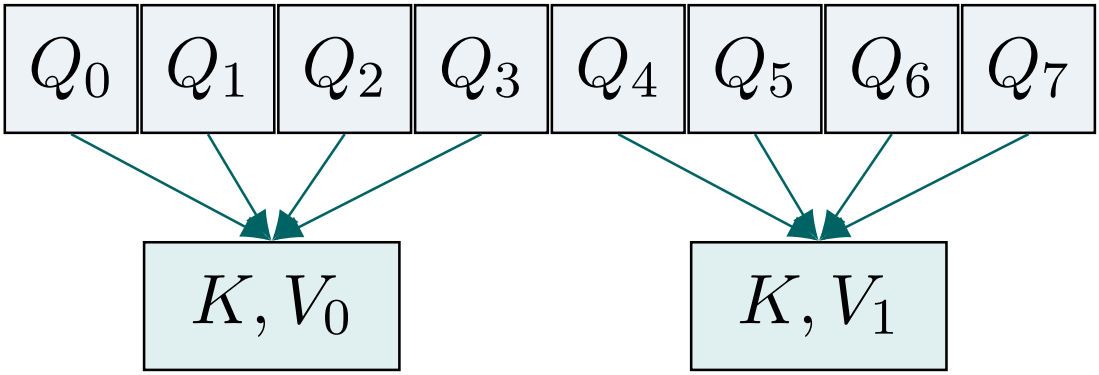

Figure 40.1: GQA maps groups of query heads to fewer key/value heads.

Intuition

GQA is a compromise between MHA quality and MQA bandwidth.

Mathematical or contract statement

If $g = H_q/H_{kv}$ is an integer, query head h reads key/value head $\lfloor h/g \rfloor$. The cache stores H_{kv} heads, while the attention output still has H_q query-head outputs.

Implementation interpretation

Validate grouping with small distinguishable heads.

How to verify this works

- Check shapes and dtypes at the boundary.
- Compare deterministic outputs against PyTorch golden vectors.
- Record tolerance, seed, model artifact, and backend.
- Do not benchmark until validation passes.

Common failure modes

- Letting framework defaults choose dtype or layout.
- Testing only a batch size of one.
- Depending on upstream checkpoint names in runtime code.

- Treating benchmark output as validation.

Exercises

E40.1. State the central invariant in one sentence.
Hint. Use the conceptual guide vocabulary.

E40.2. Construct a toy example with small shapes.
Hint. Use $B = 2$, $T = 3$, $H = 8$ where possible.

E40.3. Name a validation test that would catch a plausible bug.
Hint. Prefer generated golden vectors to handwritten long arrays.

Chapter 41

Sliding-window attention and cache policies

Sliding-window attention limits visible history and changes cache semantics. [Jiang et al., 2023]

$$\text{visible}(t, W) = \{\max(0, t + 1 - W), \ldots, t\}.$$

Intuition

A cache policy is model semantics.

Mathematical or contract statement

For query position t, windowed attention exposes only the bounded suffix $\text{visible}(t, W)$. Append-only attention is the special case where W is at least the full accumulated sequence length.

Implementation interpretation

Keep phase-0 append-only; add sliding windows as a later policy.

How to verify this works

- Check shapes and dtypes at the boundary.
- Compare deterministic outputs against PyTorch golden vectors.
- Record tolerance, seed, model artifact, and backend.
- Do not benchmark until validation passes.

Common failure modes

- Letting framework defaults choose dtype or layout.
- Testing only a batch size of one.
- Depending on upstream checkpoint names in runtime code.
- Treating benchmark output as validation.

Exercises

E41.1. State the central invariant in one sentence.
Hint. Use the conceptual guide vocabulary.

E41.2. Construct a toy example with small shapes.

Hint. Use $B = 2$, $T = 3$, $H = 8$ where possible.

E41.3. Name a validation test that would catch a plausible bug.

Hint. Prefer generated golden vectors to handwritten long arrays.

Chapter 42

Modern dense model-family adapters

Modern dense families differ in norm, attention, tokenizer, position, and checkpoint naming details.

Intuition

Adapters translate a family into internal contracts.

Mathematical or contract statement

A family adapter supplies config rules, tensor mappings, and validation plans.

Implementation interpretation

Keep adapters thin so core runtime remains reusable.

How to verify this works

- Check shapes and dtypes at the boundary.
- Compare deterministic outputs against PyTorch golden vectors.
- Record tolerance, seed, model artifact, and backend.
- Do not benchmark until validation passes.

Common failure modes

- Letting framework defaults choose dtype or layout.
- Testing only a batch size of one.
- Depending on upstream checkpoint names in runtime code.
- Treating benchmark output as validation.

Exercises

E42.1. State the central invariant in one sentence.
Hint. Use the conceptual guide vocabulary.

E42.2. Construct a toy example with small shapes.
Hint. Use $B = 2$, $T = 3$, $H = 8$ where possible.

E42.3. Name a validation test that would catch a plausible bug.
Hint. Prefer generated golden vectors to handwritten long arrays.

Chapter 43

Qwen/Gemma/Mistral-style implementation considerations

Qwen, Gemma, and Mistral-style models illustrate why family adapters are necessary. [Yang et al., 2024, Gemma Team, 2024, Jiang et al., 2023]

Intuition

Model-family names are not architecture contracts.

Mathematical or contract statement

The common core stays decoder inference; adapters specify variant details.

Implementation interpretation

Do not claim support without converter tests and golden vectors.

How to verify this works

- Check shapes and dtypes at the boundary.
- Compare deterministic outputs against PyTorch golden vectors.
- Record tolerance, seed, model artifact, and backend.
- Do not benchmark until validation passes.

Common failure modes

- Letting framework defaults choose dtype or layout.
- Testing only a batch size of one.
- Depending on upstream checkpoint names in runtime code.
- Treating benchmark output as validation.

Exercises

E43.1. State the central invariant in one sentence.
Hint. Use the conceptual guide vocabulary.

E43.2. Construct a toy example with small shapes.
Hint. Use $B = 2$, $T = 3$, $H = 8$ where possible.

E43.3. Name a validation test that would catch a plausible bug.
Hint. Prefer generated golden vectors to handwritten long arrays.

Chapter 44

Long-context validation

Long-context validation stresses position offsets, cache memory, visibility policies, and tolerance accumulation.

Intuition

Context length is a semantic claim and a systems claim.

Mathematical or contract statement

Long-context tests cover positions near limits, cache lengths, memory estimates, and decode equivalence.

Implementation interpretation

Use generated vectors and summaries rather than hard-coded long arrays.

How to verify this works

- Check shapes and dtypes at the boundary.
- Compare deterministic outputs against PyTorch golden vectors.
- Record tolerance, seed, model artifact, and backend.
- Do not benchmark until validation passes.

Common failure modes

- Letting framework defaults choose dtype or layout.
- Testing only a batch size of one.
- Depending on upstream checkpoint names in runtime code.
- Treating benchmark output as validation.

Exercises

E44.1. State the central invariant in one sentence.
Hint. Use the conceptual guide vocabulary.

E44.2. Construct a toy example with small shapes.
Hint. Use $B = 2$, $T = 3$, $H = 8$ where possible.

E44.3. Name a validation test that would catch a plausible bug.
Hint. Prefer generated golden vectors to handwritten long arrays.

Chapter 45

Quantisation overview, marked as later work

Quantisation reduces memory bandwidth and storage but is later work relative to phase-0 float32 correctness.

Intuition

Quantisation changes the numerical contract.

Mathematical or contract statement

Quantised inference requires scale or block metadata and a documented comparison tolerance.

Implementation interpretation

Do not let quantisation leak into phase-0 artifacts.

How to verify this works

- Check shapes and dtypes at the boundary.
- Compare deterministic outputs against PyTorch golden vectors.
- Record tolerance, seed, model artifact, and backend.
- Do not benchmark until validation passes.

Common failure modes

- Letting framework defaults choose dtype or layout.
- Testing only a batch size of one.
- Depending on upstream checkpoint names in runtime code.
- Treating benchmark output as validation.

Exercises

E45.1. State the central invariant in one sentence.
Hint. Use the conceptual guide vocabulary.

E45.2. Construct a toy example with small shapes.
Hint. Use $B = 2$, $T = 3$, $H = 8$ where possible.

E45.3. Name a validation test that would catch a plausible bug.
Hint. Prefer generated golden vectors to handwritten long arrays.

Part V

Hybrid and recurrent phases

Chapter 46

Why hybrid token mixers matter

Hybrid token mixers explore architectures beyond pure attention.

Intuition

Attention stores history explicitly; recurrent mixers compress it into state.

Mathematical or contract statement

A heterogeneous stack may contain attention blocks, MLPs, and recurrent blocks, each with its own state.

Implementation interpretation

Let layers declare state kind: KV cache, recurrent state, both, or neither.

How to verify this works

- Check shapes and dtypes at the boundary.
- Compare deterministic outputs against PyTorch golden vectors.
- Record tolerance, seed, model artifact, and backend.
- Do not benchmark until validation passes.

Common failure modes

- Letting framework defaults choose dtype or layout.
- Testing only a batch size of one.
- Depending on upstream checkpoint names in runtime code.
- Treating benchmark output as validation.

Exercises

E46.1. State the central invariant in one sentence.
Hint. Use the conceptual guide vocabulary.

E46.2. Construct a toy example with small shapes.
Hint. Use $B = 2$, $T = 3$, $H = 8$ where possible.

E46.3. Name a validation test that would catch a plausible bug.
Hint. Prefer generated golden vectors to handwritten long arrays.

Chapter 47

State-space and recurrent intuition

State-space and recurrent models update hidden state as tokens arrive. [Gu and Dao, 2024]

$$h_t = A_t h_{t-1} + B_t x_t, \qquad y_t = C_t h_t + D_t x_t,$$

where h_t is recurrent state, x_t is the token representation, and the matrices may be input-dependent in selective state-space models.

Intuition

A recurrent state is a summary with an update law.

Mathematical or contract statement

At a high level, a recurrent token mixer maps (h_{t-1}, x_t) to (h_t, y_t). Sequence-mode evaluation and repeated step-mode evaluation must produce matching $y_{1:T}$ within tolerance.

Implementation interpretation

Validate sequence scan against repeated step updates.

How to verify this works

- Check shapes and dtypes at the boundary.
- Compare deterministic outputs against PyTorch golden vectors.
- Record tolerance, seed, model artifact, and backend.
- Do not benchmark until validation passes.

Common failure modes

- Letting framework defaults choose dtype or layout.
- Testing only a batch size of one.
- Depending on upstream checkpoint names in runtime code.
- Treating benchmark output as validation.

Exercises

E47.1. State the central invariant in one sentence.
Hint. Use the conceptual guide vocabulary.

E47.2. Construct a toy example with small shapes.

Hint. Use $B = 2$, $T = 3$, $H = 8$ where possible.

E47.3. Name a validation test that would catch a plausible bug.

Hint. Prefer generated golden vectors to handwritten long arrays.

Chapter 48

Mamba-style blocks

Mamba-style blocks are a concrete later-phase recurrent or state-space family. [Gu and Dao, 2024]

Intuition

A Mamba-style block replaces all-pairs attention with a structured scan.

Mathematical or contract statement

The exact parameterisation is model-specific, but the engine abstraction can be common.

Implementation interpretation

Add `MambaBlock` with state allocation, scan, step update, and exports.

How to verify this works

- Check shapes and dtypes at the boundary.
- Compare deterministic outputs against PyTorch golden vectors.
- Record tolerance, seed, model artifact, and backend.
- Do not benchmark until validation passes.

Common failure modes

- Letting framework defaults choose dtype or layout.
- Testing only a batch size of one.
- Depending on upstream checkpoint names in runtime code.
- Treating benchmark output as validation.

Exercises

E48.1. State the central invariant in one sentence.
Hint. Use the conceptual guide vocabulary.

E48.2. Construct a toy example with small shapes.
Hint. Use $B = 2$, $T = 3$, $H = 8$ where possible.

E48.3. Name a validation test that would catch a plausible bug.
Hint. Prefer generated golden vectors to handwritten long arrays.

Chapter 49

Recurrent state versus KV-cache

KV cache and recurrent state are both generation state but have different semantics.

Intuition

The type of state determines the equivalence law.

Mathematical or contract statement

KV cache equivalence compares full attention recomputation with cached decode; recurrent equivalence compares scan with repeated step updates.

Implementation interpretation

Use separate tagged state variants.

How to verify this works

- Check shapes and dtypes at the boundary.
- Compare deterministic outputs against PyTorch golden vectors.
- Record tolerance, seed, model artifact, and backend.
- Do not benchmark until validation passes.

Common failure modes

- Letting framework defaults choose dtype or layout.
- Testing only a batch size of one.
- Depending on upstream checkpoint names in runtime code.
- Treating benchmark output as validation.

Exercises

E49.1. State the central invariant in one sentence.
Hint. Use the conceptual guide vocabulary.

E49.2. Construct a toy example with small shapes.
Hint. Use $B = 2$, $T = 3$, $H = 8$ where possible.

E49.3. Name a validation test that would catch a plausible bug.
Hint. Prefer generated golden vectors to handwritten long arrays.

Chapter 50

Interface design for heterogeneous layers

Heterogeneous layers require an interface that declares parameters, state kind, prefill behaviour, decode behaviour, and validation exports.

Intuition

Heterogeneity should be explicit, not hidden in a switch statement.

Mathematical or contract statement

A layer maps $(x, state)$ to $(y, state')$ and may expose diagnostics.

Implementation interpretation

Keep forward, state allocation, state clear, and profiling labels narrow.

How to verify this works

- Check shapes and dtypes at the boundary.
- Compare deterministic outputs against PyTorch golden vectors.
- Record tolerance, seed, model artifact, and backend.
- Do not benchmark until validation passes.

Common failure modes

- Letting framework defaults choose dtype or layout.
- Testing only a batch size of one.
- Depending on upstream checkpoint names in runtime code.
- Treating benchmark output as validation.

Exercises

E50.1. State the central invariant in one sentence.
Hint. Use the conceptual guide vocabulary.

E50.2. Construct a toy example with small shapes.
Hint. Use $B = 2$, $T = 3$, $H = 8$ where possible.

E50.3. Name a validation test that would catch a plausible bug.
Hint. Prefer generated golden vectors to handwritten long arrays.

Chapter 51

Validation for hybrid models

Hybrid validation extends transformer validation with state-update checks.

Intuition

Each new state semantics needs its own equivalence law.

Mathematical or contract statement

Validate component outputs, full logits, recurrent scan equivalence, attention cache equivalence, and greedy ids.

Implementation interpretation

Start with tiny synthetic configs and generated vectors.

How to verify this works

- Check shapes and dtypes at the boundary.
- Compare deterministic outputs against PyTorch golden vectors.
- Record tolerance, seed, model artifact, and backend.
- Do not benchmark until validation passes.

Common failure modes

- Letting framework defaults choose dtype or layout.
- Testing only a batch size of one.
- Depending on upstream checkpoint names in runtime code.
- Treating benchmark output as validation.

Exercises

E51.1. State the central invariant in one sentence.
Hint. Use the conceptual guide vocabulary.

E51.2. Construct a toy example with small shapes.
Hint. Use $B = 2$, $T = 3$, $H = 8$ where possible.

E51.3. Name a validation test that would catch a plausible bug.
Hint. Prefer generated golden vectors to handwritten long arrays.

Part VI

CUDA acceleration and performance

Chapter 52

From CPU kernels to CUDA kernels

CUDA acceleration begins from a CPU-correct tensor contract. Exact API details should be checked against official CUDA documentation for the installed toolkit. [NVIDIA Corporation, 2026d,b,a]

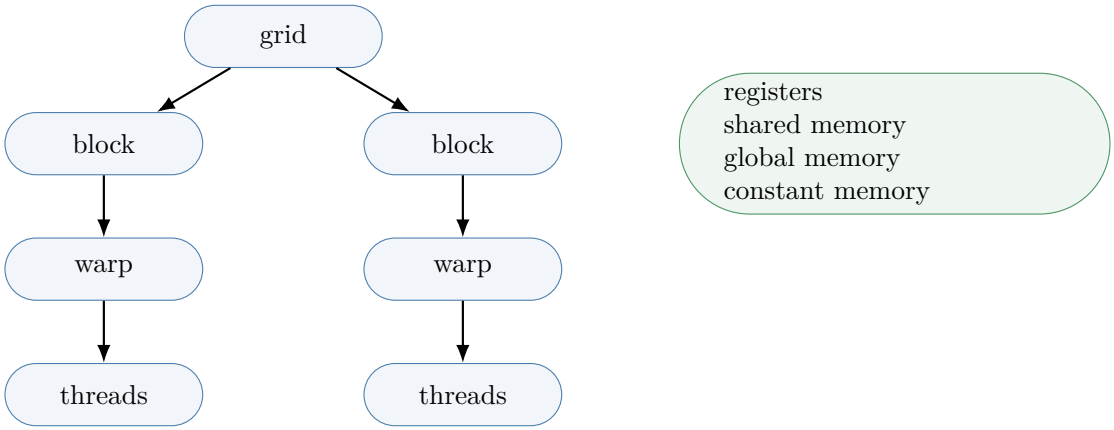

Figure 52.1: CUDA execution and memory hierarchy.

Concept	Engine meaning
Thread	Scalar work lane.
Warp	Scheduling group; coalescing and divergence matter.
Block	Cooperative group with shared memory.
Stream	Ordered queue of GPU work.
Event	Timing and dependency marker.
CUDA Graph	Captured repeated launch graph.

Table 52.1: CUDA execution checklist.

Intuition

A GPU kernel is a parallel implementation of an already proven operation.

Mathematical or contract statement

CUDA executes grids of blocks of threads, scheduled in warps.

Implementation interpretation

Compare every CUDA output against CPU/PyTorch golden vectors before profiling.

How to verify this works

- Check shapes and dtypes at the boundary.
- Compare deterministic outputs against PyTorch golden vectors.
- Record tolerance, seed, model artifact, and backend.
- Do not benchmark until validation passes.

Common failure modes

- Letting framework defaults choose dtype or layout.
- Testing only a batch size of one.
- Depending on upstream checkpoint names in runtime code.
- Treating benchmark output as validation.

Exercises

E52.1. State the central invariant in one sentence.
Hint. Use the conceptual guide vocabulary.

E52.2. Construct a toy example with small shapes.
Hint. Use $B = 2$, $T = 3$, $H = 8$ where possible.

E52.3. Name a validation test that would catch a plausible bug.
Hint. Prefer generated golden vectors to handwritten long arrays.

Chapter 53

Memory layout and tensor storage

GPU performance is often dominated by memory traffic, so layout decisions matter.

Memory	Engine concern
Registers	Per-thread temporaries and spill risk.
Shared memory	Block-level staging and reductions.
Global memory	DRAM bandwidth and coalescing.
Constant memory	Small broadcast-friendly values.
Host memory	Transfers, pinning, and orchestration.

Table 53.1: Memory hierarchy as a design checklist.

Intuition

Layout is where algebra becomes bandwidth.

Mathematical or contract statement

Artifact row-major layout is a serialisation contract; GPU internal layout may differ if logical indexing is preserved.

Implementation interpretation

Repack weights only after validating equality with artifact tensors.

How to verify this works

- Check shapes and dtypes at the boundary.
- Compare deterministic outputs against PyTorch golden vectors.
- Record tolerance, seed, model artifact, and backend.
- Do not benchmark until validation passes.

Common failure modes

- Letting framework defaults choose dtype or layout.
- Testing only a batch size of one.
- Depending on upstream checkpoint names in runtime code.
- Treating benchmark output as validation.

Exercises

E53.1. State the central invariant in one sentence.
Hint. Use the conceptual guide vocabulary.

E53.2. Construct a toy example with small shapes.
Hint. Use $B = 2$, $T = 3$, $H = 8$ where possible.

E53.3. Name a validation test that would catch a plausible bug.
Hint. Prefer generated golden vectors to handwritten long arrays.

Chapter 54

CUDA matmul concepts

Matmul dominates projection and MLP work. CUTLASS-style ideas use tiling, shared memory, and Tensor Cores. [NVIDIA Corporation, 2026e]

$$\text{arithmetic intensity} \approx \frac{2MNK}{\text{bytes read and written}}.$$

Intuition

Matmul speed comes from reusing data after reading it.

Mathematical or contract statement

For $A \in \mathbb{R}^{M\times K}$ and $B \in \mathbb{R}^{K\times N}$, computing $C = AB$ performs MNK multiply-adds, commonly counted as roughly $2MNK$ floating-point operations.

Implementation interpretation

Prefer trusted libraries for production while using toy kernels for education.

How to verify this works

- Check shapes and dtypes at the boundary.
- Compare deterministic outputs against PyTorch golden vectors.
- Record tolerance, seed, model artifact, and backend.
- Do not benchmark until validation passes.

Common failure modes

- Letting framework defaults choose dtype or layout.
- Testing only a batch size of one.
- Depending on upstream checkpoint names in runtime code.
- Treating benchmark output as validation.

Exercises

E54.1. State the central invariant in one sentence.
Hint. Use the conceptual guide vocabulary.

E54.2. Construct a toy example with small shapes.
Hint. Use $B = 2$, $T = 3$, $H = 8$ where possible.

E54.3. Name a validation test that would catch a plausible bug.

Hint. Prefer generated golden vectors to handwritten long arrays.

Chapter 55

LayerNorm and RMSNorm kernels

Normalisation kernels are row-wise reductions over hidden size.

Intuition

A norm kernel is a reduction plus an elementwise transform.

Mathematical or contract statement

LayerNorm reduces mean and variance; RMSNorm reduces mean square.

Implementation interpretation

Map rows to warps or blocks and validate epsilon and axis semantics.

How to verify this works

- Check shapes and dtypes at the boundary.
- Compare deterministic outputs against PyTorch golden vectors.
- Record tolerance, seed, model artifact, and backend.
- Do not benchmark until validation passes.

Common failure modes

- Letting framework defaults choose dtype or layout.
- Testing only a batch size of one.
- Depending on upstream checkpoint names in runtime code.
- Treating benchmark output as validation.

Exercises

E55.1. State the central invariant in one sentence.
Hint. Use the conceptual guide vocabulary.

E55.2. Construct a toy example with small shapes.
Hint. Use $B = 2$, $T = 3$, $H = 8$ where possible.

E55.3. Name a validation test that would catch a plausible bug.
Hint. Prefer generated golden vectors to handwritten long arrays.

Chapter 56

Softmax kernels

Softmax appears in attention and sampling.

A stable softmax kernel performs a maximum reduction, an exponential sum reduction after max subtraction, and normalisation. Fused attention avoids materialising large probability matrices. [Dao et al., 2022, Dao, 2024]

Intuition

Stable softmax is a reduction pipeline.

Mathematical or contract statement

Subtract the maximum, sum exponentials, and normalise, applying masks before normalisation.

Implementation interpretation

Validate shift invariance and masked support on GPU.

How to verify this works

- Check shapes and dtypes at the boundary.
- Compare deterministic outputs against PyTorch golden vectors.
- Record tolerance, seed, model artifact, and backend.
- Do not benchmark until validation passes.

Common failure modes

- Letting framework defaults choose dtype or layout.
- Testing only a batch size of one.
- Depending on upstream checkpoint names in runtime code.
- Treating benchmark output as validation.

Exercises

E56.1. State the central invariant in one sentence.
Hint. Use the conceptual guide vocabulary.

E56.2. Construct a toy example with small shapes.
Hint. Use $B = 2$, $T = 3$, $H = 8$ where possible.

E56.3. Name a validation test that would catch a plausible bug.

Hint. Prefer generated golden vectors to handwritten long arrays.

Chapter 57

Attention kernels

Attention kernels combine score computation, masking, softmax, and value mixing. [Dao et al., 2022, Dao, 2024]

Intuition

The costly object is often the intermediate matrix.

Mathematical or contract statement

Exact attention can stream tiles while maintaining stable softmax statistics.

Implementation interpretation

Keep unfused and fused paths comparable.

How to verify this works

- Check shapes and dtypes at the boundary.
- Compare deterministic outputs against PyTorch golden vectors.
- Record tolerance, seed, model artifact, and backend.
- Do not benchmark until validation passes.

Common failure modes

- Letting framework defaults choose dtype or layout.
- Testing only a batch size of one.
- Depending on upstream checkpoint names in runtime code.
- Treating benchmark output as validation.

Exercises

E57.1. State the central invariant in one sentence.
Hint. Use the conceptual guide vocabulary.

E57.2. Construct a toy example with small shapes.
Hint. Use $B = 2$, $T = 3$, $H = 8$ where possible.

E57.3. Name a validation test that would catch a plausible bug.
Hint. Prefer generated golden vectors to handwritten long arrays.

Chapter 58

Prefill kernels

Prefill processes prompt sequences with enough parallel work to keep the GPU busy.

Intuition

Prefill is throughput-oriented.

Mathematical or contract statement

Prefill writes cache entries for all prompt positions.

Implementation interpretation

Benchmark prefill separately from decode.

How to verify this works

- Check shapes and dtypes at the boundary.
- Compare deterministic outputs against PyTorch golden vectors.
- Record tolerance, seed, model artifact, and backend.
- Do not benchmark until validation passes.

Common failure modes

- Letting framework defaults choose dtype or layout.
- Testing only a batch size of one.
- Depending on upstream checkpoint names in runtime code.
- Treating benchmark output as validation.

Exercises

E58.1. State the central invariant in one sentence.
Hint. Use the conceptual guide vocabulary.

E58.2. Construct a toy example with small shapes.
Hint. Use $B = 2$, $T = 3$, $H = 8$ where possible.

E58.3. Name a validation test that would catch a plausible bug.
Hint. Prefer generated golden vectors to handwritten long arrays.

Chapter 59

Decode kernels

Decode often processes one token and reads the accumulated KV cache.

Intuition

Decode is latency and bandwidth sensitive.

Mathematical or contract statement

For prompt length m and zero-based decode step t, each layer reads visible length $S = m + t$ before appending and writes one new key/value position, giving cache length $S + 1$ after the step.

Implementation interpretation

Use decode-specific kernels and validate cache equivalence.

How to verify this works

- Check shapes and dtypes at the boundary.
- Compare deterministic outputs against PyTorch golden vectors.
- Record tolerance, seed, model artifact, and backend.
- Do not benchmark until validation passes.

Common failure modes

- Letting framework defaults choose dtype or layout.
- Testing only a batch size of one.
- Depending on upstream checkpoint names in runtime code.
- Treating benchmark output as validation.

Exercises

E59.1. State the central invariant in one sentence.
Hint. Use the conceptual guide vocabulary.

E59.2. Construct a toy example with small shapes.
Hint. Use $B = 2$, $T = 3$, $H = 8$ where possible.

E59.3. Name a validation test that would catch a plausible bug.
Hint. Prefer generated golden vectors to handwritten long arrays.

Chapter 60

KV-cache layout on GPU

The GPU KV cache must preserve logical layout while optimising physical memory reads.

Intuition

The cache is usually the largest live decode state.

Mathematical or contract statement

MHA, MQA, and GQA differ primarily in H_{kv}, which controls cache bytes. For element size e bytes and L layers, an append-only KV cache stores approximately $2LBH_{kv}Sd_he$ bytes.

Implementation interpretation

Keep logical cache API separate from physical allocation.

How to verify this works

- Check shapes and dtypes at the boundary.
- Compare deterministic outputs against PyTorch golden vectors.
- Record tolerance, seed, model artifact, and backend.
- Do not benchmark until validation passes.

Common failure modes

- Letting framework defaults choose dtype or layout.
- Testing only a batch size of one.
- Depending on upstream checkpoint names in runtime code.
- Treating benchmark output as validation.

Exercises

E60.1. State the central invariant in one sentence.
Hint. Use the conceptual guide vocabulary.

E60.2. Construct a toy example with small shapes.
Hint. Use $B = 2$, $T = 3$, $H = 8$ where possible.

E60.3. Name a validation test that would catch a plausible bug.
Hint. Prefer generated golden vectors to handwritten long arrays.

Chapter 61

Kernel fusion

Kernel fusion reduces memory traffic and launch overhead by combining operations.

Intuition

Fusion should remove memory movement, not validation boundaries.

Mathematical or contract statement

Examples include residual plus norm, bias plus activation, and fused attention.

Implementation interpretation

Keep unfused references for debugging.

How to verify this works

- Check shapes and dtypes at the boundary.
- Compare deterministic outputs against PyTorch golden vectors.
- Record tolerance, seed, model artifact, and backend.
- Do not benchmark until validation passes.

Common failure modes

- Letting framework defaults choose dtype or layout.
- Testing only a batch size of one.
- Depending on upstream checkpoint names in runtime code.
- Treating benchmark output as validation.

Exercises

E61.1. State the central invariant in one sentence.
Hint. Use the conceptual guide vocabulary.

E61.2. Construct a toy example with small shapes.
Hint. Use $B = 2$, $T = 3$, $H = 8$ where possible.

E61.3. Name a validation test that would catch a plausible bug.
Hint. Prefer generated golden vectors to handwritten long arrays.

Chapter 62

Tensor Cores and mixed precision

Tensor Cores accelerate supported matrix operations and dtypes. [NVIDIA Corporation, 2026d,e]

Intuition

Specialised hardware is useful only when the numerical contract permits it.

Mathematical or contract statement

Float16 and bfloat16 reduce bandwidth but change tolerance expectations.

Implementation interpretation

Keep phase-0 float32 artifact support separate from later mixed-precision execution.

How to verify this works

- Check shapes and dtypes at the boundary.
- Compare deterministic outputs against PyTorch golden vectors.
- Record tolerance, seed, model artifact, and backend.
- Do not benchmark until validation passes.

Common failure modes

- Letting framework defaults choose dtype or layout.
- Testing only a batch size of one.
- Depending on upstream checkpoint names in runtime code.
- Treating benchmark output as validation.

Exercises

E62.1. State the central invariant in one sentence.
Hint. Use the conceptual guide vocabulary.

E62.2. Construct a toy example with small shapes.
Hint. Use $B = 2$, $T = 3$, $H = 8$ where possible.

E62.3. Name a validation test that would catch a plausible bug.
Hint. Prefer generated golden vectors to handwritten long arrays.

Chapter 63

CUDA Graphs and launch overhead

CUDA Graphs are relevant to repeated decode schedules. [NVIDIA Corporation, 2026c]

CUDA Graphs can reduce repeated launch overhead, but only after output equality and shape/memory stability are validated. [NVIDIA Corporation, 2026c]

Intuition

A repeated schedule can be captured and replayed.

Mathematical or contract statement

A graph captures operations and dependencies, reducing launch overhead if shapes and memory addresses are stable.

Implementation interpretation

Validate non-graph and graph outputs before trusting speed.

How to verify this works

- Check shapes and dtypes at the boundary.
- Compare deterministic outputs against PyTorch golden vectors.
- Record tolerance, seed, model artifact, and backend.
- Do not benchmark until validation passes.

Common failure modes

- Letting framework defaults choose dtype or layout.
- Testing only a batch size of one.
- Depending on upstream checkpoint names in runtime code.
- Treating benchmark output as validation.

Exercises

E63.1. State the central invariant in one sentence.
Hint. Use the conceptual guide vocabulary.

E63.2. Construct a toy example with small shapes.
Hint. Use $B = 2$, $T = 3$, $H = 8$ where possible.

E63.3. Name a validation test that would catch a plausible bug.
Hint. Prefer generated golden vectors to handwritten long arrays.

Chapter 64

Profiling workflow

Profiling turns performance guesses into measurements. Nsight Compute and CUDA events are common tools. [NVIDIA Corporation, 2026f,d]

Intuition

A profiler explains why, not just how fast.

Mathematical or contract statement

Useful diagnostics include memory throughput, warp stalls, Tensor Core utilisation, occupancy, and launch overhead.

Implementation interpretation

Profile only after correctness passes.

How to verify this works

- Check shapes and dtypes at the boundary.
- Compare deterministic outputs against PyTorch golden vectors.
- Record tolerance, seed, model artifact, and backend.
- Do not benchmark until validation passes.

Common failure modes

- Letting framework defaults choose dtype or layout.
- Testing only a batch size of one.
- Depending on upstream checkpoint names in runtime code.
- Treating benchmark output as validation.

Exercises

E64.1. State the central invariant in one sentence.
Hint. Use the conceptual guide vocabulary.

E64.2. Construct a toy example with small shapes.
Hint. Use $B = 2$, $T = 3$, $H = 8$ where possible.

E64.3. Name a validation test that would catch a plausible bug.
Hint. Prefer generated golden vectors to handwritten long arrays.

Chapter 65

Benchmark interpretation

Benchmark interpretation separates prefill throughput, decode latency, first-token latency, and end-to-end generation.

Intuition

A benchmark number needs a workload.

Mathematical or contract statement

Throughput, latency, bandwidth, and arithmetic intensity measure different behaviours.

Implementation interpretation

Report prompt length, decode length, batch size, backend, and validation status.

How to verify this works

- Check shapes and dtypes at the boundary.
- Compare deterministic outputs against PyTorch golden vectors.
- Record tolerance, seed, model artifact, and backend.
- Do not benchmark until validation passes.

Common failure modes

- Letting framework defaults choose dtype or layout.
- Testing only a batch size of one.
- Depending on upstream checkpoint names in runtime code.
- Treating benchmark output as validation.

Exercises

E65.1. State the central invariant in one sentence.
Hint. Use the conceptual guide vocabulary.

E65.2. Construct a toy example with small shapes.
Hint. Use $B = 2$, $T = 3$, $H = 8$ where possible.

E65.3. Name a validation test that would catch a plausible bug.
Hint. Prefer generated golden vectors to handwritten long arrays.

Chapter 66

Optional architecture notes for RTX 40-series and RTX 5060-style GPUs

The target is generic NVIDIA CUDA GPUs; consumer GPUs are useful development targets but not design foundations. [NVIDIA Corporation, 2026g,h,d]

Intuition

Portable performance starts from capability queries.

Mathematical or contract statement

Query compute capability, memory size, dtype support, shared memory limits, and graph features at runtime.

Implementation interpretation

Use architecture-specific paths only behind fallbacks.

How to verify this works

- Check shapes and dtypes at the boundary.
- Compare deterministic outputs against PyTorch golden vectors.
- Record tolerance, seed, model artifact, and backend.
- Do not benchmark until validation passes.

Common failure modes

- Letting framework defaults choose dtype or layout.
- Testing only a batch size of one.
- Depending on upstream checkpoint names in runtime code.
- Treating benchmark output as validation.

Exercises

E66.1. State the central invariant in one sentence.
Hint. Use the conceptual guide vocabulary.

E66.2. Construct a toy example with small shapes.
Hint. Use $B = 2$, $T = 3$, $H = 8$ where possible.

E66.3. Name a validation test that would catch a plausible bug.

Hint. Prefer generated golden vectors to handwritten long arrays.

Part VII

Training for context

Chapter 67

What training would add

Training is not the project goal, but knowing what it would add clarifies the inference focus.

Intuition

Training turns forward execution into differentiable optimisation.

Mathematical or contract statement

Training requires gradients, loss, optimiser state, dataset pipelines, checkpoint writing, and often distributed execution.

Implementation interpretation

Add backward methods, gradient buffers, optimiser hooks, and training checkpoint formats only in a separate project phase.

How to verify this works

- Check shapes and dtypes at the boundary.
- Compare deterministic outputs against PyTorch golden vectors.
- Record tolerance, seed, model artifact, and backend.
- Do not benchmark until validation passes.

Common failure modes

- Letting framework defaults choose dtype or layout.
- Testing only a batch size of one.
- Depending on upstream checkpoint names in runtime code.
- Treating benchmark output as validation.

Exercises

E67.1. State the central invariant in one sentence.
Hint. Use the conceptual guide vocabulary.

E67.2. Construct a toy example with small shapes.
Hint. Use $B = 2$, $T = 3$, $H = 8$ where possible.

E67.3. Name a validation test that would catch a plausible bug.
Hint. Prefer generated golden vectors to handwritten long arrays.

Chapter 68

Language-modelling loss

The standard autoregressive training objective is next-token prediction.

$$\mathcal{L}(\theta) = -\sum_{t=1}^{T-1} \log p_\theta(x_{t+1} \mid x_{\leq t}).$$

Intuition

The model is penalised for assigning low probability to the observed next token.

Mathematical or contract statement

The cross-entropy loss sums negative log probabilities of next tokens.

Implementation interpretation

Training implementation would shift logits and labels, mask padding, and compute stable log-softmax.

How to verify this works

- Check shapes and dtypes at the boundary.
- Compare deterministic outputs against PyTorch golden vectors.
- Record tolerance, seed, model artifact, and backend.
- Do not benchmark until validation passes.

Common failure modes

- Letting framework defaults choose dtype or layout.
- Testing only a batch size of one.
- Depending on upstream checkpoint names in runtime code.
- Treating benchmark output as validation.

Exercises

E68.1. State the central invariant in one sentence.
Hint. Use the conceptual guide vocabulary.

E68.2. Construct a toy example with small shapes.
Hint. Use $B = 2$, $T = 3$, $H = 8$ where possible.

E68.3. Name a validation test that would catch a plausible bug.
Hint. Prefer generated golden vectors to handwritten long arrays.

Chapter 69

Backpropagation overview

Backpropagation computes derivatives of loss with respect to parameters and activations.

Intuition

Backward pass reverses the forward graph.

Mathematical or contract statement

For $y = f(g(x))$, gradients use the chain rule.

Implementation interpretation

A training engine must store or recompute activations and implement backward kernels.

How to verify this works

- Check shapes and dtypes at the boundary.
- Compare deterministic outputs against PyTorch golden vectors.
- Record tolerance, seed, model artifact, and backend.
- Do not benchmark until validation passes.

Common failure modes

- Letting framework defaults choose dtype or layout.
- Testing only a batch size of one.
- Depending on upstream checkpoint names in runtime code.
- Treating benchmark output as validation.

Exercises

E69.1. State the central invariant in one sentence.
Hint. Use the conceptual guide vocabulary.

E69.2. Construct a toy example with small shapes.
Hint. Use $B = 2$, $T = 3$, $H = 8$ where possible.

E69.3. Name a validation test that would catch a plausible bug.
Hint. Prefer generated golden vectors to handwritten long arrays.

Chapter 70

Optimisers and state

Optimisers turn gradients into parameter updates and often keep additional state.

Intuition

Optimiser state is another model-sized system.

Mathematical or contract statement

SGD is $\theta \leftarrow \theta - \eta \nabla_\theta L$; adaptive methods store moving statistics.

Implementation interpretation

Training support would require optimiser-state checkpoints separate from inference artifacts.

How to verify this works

- Check shapes and dtypes at the boundary.
- Compare deterministic outputs against PyTorch golden vectors.
- Record tolerance, seed, model artifact, and backend.
- Do not benchmark until validation passes.

Common failure modes

- Letting framework defaults choose dtype or layout.
- Testing only a batch size of one.
- Depending on upstream checkpoint names in runtime code.
- Treating benchmark output as validation.

Exercises

E70.1. State the central invariant in one sentence.
Hint. Use the conceptual guide vocabulary.

E70.2. Construct a toy example with small shapes.
Hint. Use $B = 2$, $T = 3$, $H = 8$ where possible.

E70.3. Name a validation test that would catch a plausible bug.
Hint. Prefer generated golden vectors to handwritten long arrays.

Chapter 71

Why inference is a different engineering problem

Inference focuses on artifact loading, forward computation, cache state, generation, latency, throughput, and reproducibility.

Intuition

Inference is not simpler; it is differently constrained.

Mathematical or contract statement

A good inference engine guarantees semantics, stable loading, cache correctness, and measured performance.

Implementation interpretation

This project focuses on inference to complete the cross-language contract before training complexity.

How to verify this works

- Check shapes and dtypes at the boundary.
- Compare deterministic outputs against PyTorch golden vectors.
- Record tolerance, seed, model artifact, and backend.
- Do not benchmark until validation passes.

Common failure modes

- Letting framework defaults choose dtype or layout.
- Testing only a batch size of one.
- Depending on upstream checkpoint names in runtime code.
- Treating benchmark output as validation.

Exercises

E71.1. State the central invariant in one sentence.
Hint. Use the conceptual guide vocabulary.

E71.2. Construct a toy example with small shapes.
Hint. Use $B = 2$, $T = 3$, $H = 8$ where possible.

E71.3. Name a validation test that would catch a plausible bug.
Hint. Prefer generated golden vectors to handwritten long arrays.

Part VIII

Appendices

Appendix A

LaTeX notation reference

This appendix summarises notation used in equations and implementation checks.

Table A.1: Core notation.

Symbol	Meaning
B, T, S, L	Batch size, current input length, visible cache length, and number of layers.
V, P, H, F	Vocabulary size, maximum position length, hidden size, and feed-forward width.
H_q, H_{kv}, d_h, g	Query heads, key/value heads, per-head dimension, and GQA group size $g = H_q/H_{kv}$.
I	Token-id tensor in $\mathbb{N}^{B\times T}$.
$X^{(\ell)}$	Hidden states entering block ℓ, with shape $[B, T, H]$.
K_ℓ, V_ℓ	Layer-ℓ KV-cache tensors with logical shape $[B, H_{kv}, S, d_h]$.
Z	Logits with shape $[B, T, V]$.

Intuition

Notation reduces ambiguity.

Mathematical or contract statement

The same symbols are used in formulas, tensor shape tables, and validators.

Implementation interpretation

Update notation before adding a new module.

How to verify this works

- Check shapes and dtypes at the boundary.
- Compare deterministic outputs against PyTorch golden vectors.
- Record tolerance, seed, model artifact, and backend.
- Do not benchmark until validation passes.

Common failure modes

- Letting framework defaults choose dtype or layout.
- Testing only a batch size of one.

- Depending on upstream checkpoint names in runtime code.
- Treating benchmark output as validation.

Exercises

EA.1. State the central invariant in one sentence.
Hint. Use the conceptual guide vocabulary.

EA.2. Construct a toy example with small shapes.
Hint. Use $B = 2$, $T = 3$, $H = 8$ where possible.

EA.3. Name a validation test that would catch a plausible bug.
Hint. Prefer generated golden vectors to handwritten long arrays.

Appendix B

Tensor shape cheat sheet

This appendix collects common tensor shapes.

Table B.1: Tensor shape cheat sheet.

Tensor	Shape	Notes
Input ids	$[B, T]$	Integer token ids.
Hidden states	$[B, T, H]$	Residual stream.
Queries	$[B, H_q, T, d_h]$	Per-head query tensor.
Keys/values	$[B, H_{kv}, S, d_h]$	Cached attention state by layer.
Attention output by head	$[B, H_q, T, d_h]$	Concatenate heads back to $[B, T, H]$.
Logits	$[B, T, V]$	Vocabulary scores.
GPT-2 QKV weight	$[3H, H]$	Fused projection.
GPT-2 MLP input weight	$[F, H]$	Feed-forward expansion.
GPT-2 MLP output weight	$[H, F]$	Feed-forward contraction.

Intuition

A shape cheat sheet is a debugging tool.

Mathematical or contract statement

Phase-0 GPT-2 has $H_{kv} = H_q$; later MQA/GQA may not.

Implementation interpretation

Use the table while reviewing code for transposes.

How to verify this works

- Check shapes and dtypes at the boundary.
- Compare deterministic outputs against PyTorch golden vectors.
- Record tolerance, seed, model artifact, and backend.
- Do not benchmark until validation passes.

Common failure modes

- Letting framework defaults choose dtype or layout.
- Testing only a batch size of one.

- Depending on upstream checkpoint names in runtime code.
- Treating benchmark output as validation.

Exercises

EB.1. State the central invariant in one sentence.
Hint. Use the conceptual guide vocabulary.

EB.2. Construct a toy example with small shapes.
Hint. Use $B = 2$, $T = 3$, $H = 8$ where possible.

EB.3. Name a validation test that would catch a plausible bug.
Hint. Prefer generated golden vectors to handwritten long arrays.

Appendix C

CUDA technical details

This appendix lists CUDA details that matter for later kernels. [NVIDIA Corporation, 2026d]

Intuition

The bottleneck determines kernel design.

Mathematical or contract statement

Coalescing, occupancy, latency hiding, register pressure, shared memory, PTX, SASS, streams, events, and graphs interact.

Implementation interpretation

Use profiler evidence before architecture-specific tuning.

How to verify this works

- Check shapes and dtypes at the boundary.
- Compare deterministic outputs against PyTorch golden vectors.
- Record tolerance, seed, model artifact, and backend.
- Do not benchmark until validation passes.

Common failure modes

- Letting framework defaults choose dtype or layout.
- Testing only a batch size of one.
- Depending on upstream checkpoint names in runtime code.
- Treating benchmark output as validation.

Exercises

EC.1. State the central invariant in one sentence.
Hint. Use the conceptual guide vocabulary.

EC.2. Construct a toy example with small shapes.
Hint. Use $B = 2$, $T = 3$, $H = 8$ where possible.

EC.3. Name a validation test that would catch a plausible bug.
Hint. Prefer generated golden vectors to handwritten long arrays.

Appendix D

Rust ownership patterns for inference engines

Rust patterns include owned buffers, borrowed tensor views, typed errors, and explicit cache mutation. [Klabnik et al., 2026]

Intuition

A view is safe only when storage outlives it.

Mathematical or contract statement

Use lifetimes and typed wrappers for shapes, offsets, and byte counts.

Implementation interpretation

Keep unsafe code out of the phase-0 correctness path.

How to verify this works

- Check shapes and dtypes at the boundary.
- Compare deterministic outputs against PyTorch golden vectors.
- Record tolerance, seed, model artifact, and backend.
- Do not benchmark until validation passes.

Common failure modes

- Letting framework defaults choose dtype or layout.
- Testing only a batch size of one.
- Depending on upstream checkpoint names in runtime code.
- Treating benchmark output as validation.

Exercises

ED.1. State the central invariant in one sentence.
Hint. Use the conceptual guide vocabulary.

ED.2. Construct a toy example with small shapes.
Hint. Use $B = 2$, $T = 3$, $H = 8$ where possible.

ED.3. Name a validation test that would catch a plausible bug.
Hint. Prefer generated golden vectors to handwritten long arrays.

Appendix E

C++ memory and RAII patterns

C++ patterns include RAII file buffers, non-owning spans, and checked tensor views. [International Organization for Standardization, 2020]

Intuition

RAII makes lifetime visible.

Mathematical or contract statement

An owned buffer can safely produce tensor views if view lifetime is controlled.

Implementation interpretation

Use sanitiser builds and deterministic tests.

How to verify this works

- Check shapes and dtypes at the boundary.
- Compare deterministic outputs against PyTorch golden vectors.
- Record tolerance, seed, model artifact, and backend.
- Do not benchmark until validation passes.

Common failure modes

- Letting framework defaults choose dtype or layout.
- Testing only a batch size of one.
- Depending on upstream checkpoint names in runtime code.
- Treating benchmark output as validation.

Exercises

EE.1. State the central invariant in one sentence.
Hint. Use the conceptual guide vocabulary.

EE.2. Construct a toy example with small shapes.
Hint. Use $B = 2$, $T = 3$, $H = 8$ where possible.

EE.3. Name a validation test that would catch a plausible bug.
Hint. Prefer generated golden vectors to handwritten long arrays.

Appendix F

PyTorch reference implementation patterns

PyTorch reference patterns favour clarity, explicit dtype, and stable exports. [PyTorch Contributors, 2026]

Intuition

Reference code should be easy to distrust productively.

Mathematical or contract statement

Use internal tensor names, CPU float32, named intermediates, and deterministic vector output.

Implementation interpretation

Do not hide contracts in upstream model classes.

How to verify this works

- Check shapes and dtypes at the boundary.
- Compare deterministic outputs against PyTorch golden vectors.
- Record tolerance, seed, model artifact, and backend.
- Do not benchmark until validation passes.

Common failure modes

- Letting framework defaults choose dtype or layout.
- Testing only a batch size of one.
- Depending on upstream checkpoint names in runtime code.
- Treating benchmark output as validation.

Exercises

EF.1. State the central invariant in one sentence.
Hint. Use the conceptual guide vocabulary.

EF.2. Construct a toy example with small shapes.
Hint. Use $B = 2$, $T = 3$, $H = 8$ where possible.

EF.3. Name a validation test that would catch a plausible bug.
Hint. Prefer generated golden vectors to handwritten long arrays.

Appendix G

Artifact validation checklist

This checklist validates internal artifacts before model execution.

G.1. Parse `config.json`; reject invalid fields.
G.2. Parse `metadata.json`; check format version, endianness, dtype, and filenames.
G.3. Parse `weights.index.json`; do not rely on key order.
G.4. Check every required tensor name, dtype, shape, offset, and byte count.
G.5. Reject byte ranges that overlap or exceed `weights.bin`.
G.6. Verify phase-0 little-endian row-major `float32`.

Intuition

Bad artifacts should fail early.

Mathematical or contract statement

Validation covers config, metadata, index entries, byte ranges, dtype, endianness, and required names.

Implementation interpretation

Automate the checklist in every language.

How to verify this works

- Check shapes and dtypes at the boundary.
- Compare deterministic outputs against PyTorch golden vectors.
- Record tolerance, seed, model artifact, and backend.
- Do not benchmark until validation passes.

Common failure modes

- Letting framework defaults choose dtype or layout.
- Testing only a batch size of one.
- Depending on upstream checkpoint names in runtime code.
- Treating benchmark output as validation.

Exercises

EG.1. State the central invariant in one sentence.
Hint. Use the conceptual guide vocabulary.

EG.2. Construct a toy example with small shapes.
Hint. Use $B = 2$, $T = 3$, $H = 8$ where possible.

EG.3. Name a validation test that would catch a plausible bug.
Hint. Prefer generated golden vectors to handwritten long arrays.

Appendix H

Golden-vector generation checklist

This checklist summarises vector generation.

H.1. Select authoritative input ids.
H.2. Run the PyTorch CPU float32 reference.
H.3. Export JSON sidecar with model identity, dtype, tolerance, and check names.
H.4. Export NPZ arrays for larger tensors.
H.5. Store vectors under `artifacts/test_vectors/gpt2/gpt2/`.
H.6. Never paste long arrays into prose or comments.

Intuition

A reference vector is generated, not invented.

Mathematical or contract statement

A vector records input ids, dtype, tolerance, named arrays, and model identity.

Implementation interpretation

Regenerate vectors only after intentional reference or contract changes.

How to verify this works

- Check shapes and dtypes at the boundary.
- Compare deterministic outputs against PyTorch golden vectors.
- Record tolerance, seed, model artifact, and backend.
- Do not benchmark until validation passes.

Common failure modes

- Letting framework defaults choose dtype or layout.
- Testing only a batch size of one.
- Depending on upstream checkpoint names in runtime code.
- Treating benchmark output as validation.

Exercises

EH.1. State the central invariant in one sentence.
Hint. Use the conceptual guide vocabulary.

EH.2. Construct a toy example with small shapes.
Hint. Use $B = 2$, $T = 3$, $H = 8$ where possible.

EH.3. Name a validation test that would catch a plausible bug.
Hint. Prefer generated golden vectors to handwritten long arrays.

Appendix I

Benchmark checklist

This checklist keeps performance measurement separate from correctness.

I.1. Run validation first.
I.2. Record backend, model artifact, prompt length, decode length, and batch size.
I.3. Warm up before measuring.
I.4. Separate prefill and decode metrics.
I.5. Report enough context to reproduce.

Intuition

A benchmark is meaningful only for a specified workload.

Mathematical or contract statement

Benchmark reports should include workload, backend, and validation status.

Implementation interpretation

Use the same benchmark suite across languages.

How to verify this works

- Check shapes and dtypes at the boundary.
- Compare deterministic outputs against PyTorch golden vectors.
- Record tolerance, seed, model artifact, and backend.
- Do not benchmark until validation passes.

Common failure modes

- Letting framework defaults choose dtype or layout.
- Testing only a batch size of one.
- Depending on upstream checkpoint names in runtime code.
- Treating benchmark output as validation.

Exercises

EI.1. State the central invariant in one sentence.
Hint. Use the conceptual guide vocabulary.

EI.2. Construct a toy example with small shapes.
Hint. Use $B = 2$, $T = 3$, $H = 8$ where possible.

EI.3. Name a validation test that would catch a plausible bug.
Hint. Prefer generated golden vectors to handwritten long arrays.

Appendix J

Glossary

The glossary defines recurring terms in this project.

Artifact Internal model directory containing config, metadata, index, and binary weights.

Cache equivalence Full recomputation equals prefill plus cached decode within tolerance.

Conceptual guide Repository-level design narrative that explains intent while executable specs define exact contracts.

Decode Incremental generation path using cached state.

GQA Grouped-query attention, where H_q/H_{kv} query heads share each key/value head.

Golden vector Generated validation record consumed by all runtimes.

KV cache Per-layer keys and values with logical layout $[B, H_{kv}, S, d_h]$.

MQA Multi-query attention, the $H_{kv} = 1$ special case of grouped key/value sharing.

Prefill Prompt-processing path that initialises cache state.

RoPE Rotary positional embedding.

Tensor Core NVIDIA matrix-multiply acceleration hardware for supported operations.

Intuition

Precise words prevent imprecise code.

Mathematical or contract statement

A term is defined by its contract and verification strategy.

Implementation interpretation

Extend the glossary when new roadmap phases are added.

How to verify this works

- Check shapes and dtypes at the boundary.
- Compare deterministic outputs against PyTorch golden vectors.
- Record tolerance, seed, model artifact, and backend.
- Do not benchmark until validation passes.

Common failure modes

- Letting framework defaults choose dtype or layout.
- Testing only a batch size of one.

- Depending on upstream checkpoint names in runtime code.
- Treating benchmark output as validation.

Exercises

EJ.1. State the central invariant in one sentence.
Hint. Use the conceptual guide vocabulary.

EJ.2. Construct a toy example with small shapes.
Hint. Use $B = 2$, $T = 3$, $H = 8$ where possible.

EJ.3. Name a validation test that would catch a plausible bug.
Hint. Prefer generated golden vectors to handwritten long arrays.

Appendix K

Bibliography notes

The bibliography cites foundational papers, model references, and official documentation.

Intuition

References guide design; validation proves implementation.

Mathematical or contract statement

Use papers for architecture ideas and official docs for APIs and version-sensitive details.

Implementation interpretation

Check official CUDA, PyTorch, Rust, C++, and SafeTensors docs when versions change.

How to verify this works

- Check shapes and dtypes at the boundary.
- Compare deterministic outputs against PyTorch golden vectors.
- Record tolerance, seed, model artifact, and backend.
- Do not benchmark until validation passes.

Common failure modes

- Letting framework defaults choose dtype or layout.
- Testing only a batch size of one.
- Depending on upstream checkpoint names in runtime code.
- Treating benchmark output as validation.

Exercises

EK.1. State the central invariant in one sentence.
Hint. Use the conceptual guide vocabulary.

EK.2. Construct a toy example with small shapes.
Hint. Use $B = 2$, $T = 3$, $H = 8$ where possible.

EK.3. Name a validation test that would catch a plausible bug.
Hint. Prefer generated golden vectors to handwritten long arrays.

Bibliography

Joshua Ainslie, James Lee-Thorp, Michiel de Jong, Yury Zemlyanskiy, Federico Lebron, and Sumit Sanghai. GQA: Training generalized multi-query transformer models from multi-head checkpoints. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, 2023. URL https://aclanthology.org/2023.emnlp-main.298/.

Tri Dao. Flashattention-2: Faster attention with better parallelism and work partitioning. In *International Conference on Learning Representations*, 2024. URL https://arxiv.org/abs/2307.08691.

Tri Dao, Daniel Y. Fu, Stefano Ermon, Atri Rudra, and Christopher Re. Flashattention: Fast and memory-efficient exact attention with IO-awareness. *CoRR*, abs/2205.14135, 2022. URL https://arxiv.org/abs/2205.14135.

Gemma Team. Gemma: Open models based on gemini research and technology. *CoRR*, abs/2403.08295, 2024. URL https://arxiv.org/abs/2403.08295.

Albert Gu and Tri Dao. Mamba: Linear-time sequence modeling with selective state spaces. In *International Conference on Learning Representations*, 2024. URL https://arxiv.org/abs/2312.00752.

Hugging Face. *Safetensors Documentation*, 2026. Accessed 2026-04-28.

International Organization for Standardization. ISO/IEC 14882:2020: Programming languages – C++, 2020. C++20 standard.

Albert Q. Jiang, Alexandre Sablayrolles, Arthur Mensch, Chris Bamford, Devendra Singh Chaplot, Diego de las Casas, Florian Bressand, Gianna Lengyel, Guillaume Lample, Lucile Saulnier, Lelio Renard Lavaud, Marie-Anne Lachaux, Pierre Stock, Teven Le Scao, Thibaut Lavril, Thomas Wang, Timothee Lacroix, and William El Sayed. Mistral 7b. *CoRR*, abs/2310.06825, 2023. URL https://arxiv.org/abs/2310.06825.

Kitware. *CMake Documentation*, 2026. Accessed 2026-04-28.

Steve Klabnik, Carol Nichols, Chris Krycho, and The Rust Community. *The Rust Programming Language*, 2026. Accessed 2026-04-28.

NVIDIA Corporation. *CUDA Toolkit 13.0 Update 3 Release Notes*, 2026a. Accessed 2026-04-28.

NVIDIA Corporation. *CUDA Toolkit Archive*, 2026b. Accessed 2026-04-28.

NVIDIA Corporation. *CUDA Graphs*, 2026c. Accessed 2026-04-28.

NVIDIA Corporation. *CUDA Programming Guide*, 2026d. Accessed 2026-04-28.

NVIDIA Corporation. *CUTLASS Documentation*, 2026e. Accessed 2026-04-28.

NVIDIA Corporation. *NVIDIA Nsight Compute Documentation*, 2026f. Accessed 2026-04-28.

NVIDIA Corporation. *GeForce RTX 40 Series Graphics Cards*, 2026g. Accessed 2026-04-28.

NVIDIA Corporation. *GeForce RTX 5060 Family Graphics Cards*, 2026h. Accessed 2026-04-28.

PyTorch Contributors. *PyTorch Documentation*, 2026. Accessed 2026-04-28.

Alec Radford, Karthik Narasimhan, Tim Salimans, and Ilya Sutskever. Improving language understanding by generative pre-training. *OpenAI technical report*, 2018. URL https://cdn.openai.com/research-covers/language-unsupervised/language_understanding_paper.pdf.

Alec Radford, Jeffrey Wu, Rewon Child, David Luan, Dario Amodei, and Ilya Sutskever. Language models are unsupervised multitask learners. *OpenAI technical report*, 2019. URL https://cdn.openai.com/better-language-models/language_models_are_unsupervised_multitask_learners.pdf.

Noam Shazeer. Fast transformer decoding: One write-head is all you need. *CoRR*, abs/1911.02150, 2019. URL https://arxiv.org/abs/1911.02150.

Noam Shazeer. GLU variants improve transformer. *CoRR*, abs/2002.05202, 2020. URL https://arxiv.org/abs/2002.05202.

Jianlin Su, Yu Lu, Shengfeng Pan, Ahmed Murtadha, Bo Wen, and Yunfeng Liu. Roformer: Enhanced transformer with rotary position embedding. *CoRR*, abs/2104.09864, 2021. URL https://arxiv.org/abs/2104.09864.

The Rust Project Developers. *The Rust Edition Guide: Rust 2024*, 2025. Accessed 2026-04-28.

Hugo Touvron, Thibaut Lavril, Gautier Izacard, Xavier Martinet, Marie-Anne Lachaux, Timothee Lacroix, Baptiste Roziere, Naman Goyal, Eric Hambro, Faisal Azhar, Aurelien Rodriguez, Armand Joulin, Edouard Grave, and Guillaume Lample. LLaMA: Open and efficient foundation language models. *CoRR*, abs/2302.13971, 2023. URL https://arxiv.org/abs/2302.13971.

Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems 30*, 2017. URL https://arxiv.org/abs/1706.03762.

An Yang, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chengyuan Li, Dayiheng Liu, Fei Huang, Haoran Wei, et al. Qwen2.5 technical report. *CoRR*, abs/2412.15115, 2024. URL https://arxiv.org/abs/2412.15115.

Biao Zhang and Rico Sennrich. Root mean square layer normalization. In *Advances in Neural Information Processing Systems 32*, 2019. URL https://arxiv.org/abs/1910.07467.